



SAPIENZA
UNIVERSITÀ DI ROMA

Web and Software Architecture

UNIVERSITÀ DEGLI STUDI DI ROMA "LA
SAPIENZA"

FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
PROF. EMANUELE PANIZZI

December 21, 2024



Preface

Hello everyone!

My name is Lorenzo Sabatino and I am a student of the Computer Science course at La Sapienza University of Rome. During my academic career, I have faced numerous exams by transcribing the notes taken in class and transforming them into real study books.

I wanted to share this material with you because I believe it can be a valid help for all Computer Science students, both those who are beginners and those who are already advanced in their studies. I have written and formatted these texts with care, using the $\text{\LaTeX}$ markup language to ensure clarity and readability. I have also enriched the content with diagrams and schemes and I have integrated ideas taken from online lessons and textbooks.

However, it is important to underline that these books do not replace the classroom lessons and are not free of errors. My intention was to provide additional support for studying, but it is essential to participate in the lessons and delve into the concepts with the guidance of the teachers. Also, please note that due to the ever-evolving nature of computer science, some information may become outdated over time.

Nevertheless, I decided to share these books because I believe they can be of help to many students and I hope you will find them useful in your academic journey.

If you wish to contact me with any questions, comments or corrections, you can write to me at scrittinformatica@gmail.com. I will be happy to answer your requests.

Thank you for choosing to read these books and good luck with your studies!

Contents

Preface	1
1 Introduction	5
1.1 Web apps	5
2 Version Control with Git	7
2.1 Version Control System	7
2.2 Git	8
3 Web protocols	15
3.1 REpresentational State Transfer (REST)	15
3.2 Uniform Resource Identifier (URI)	15
3.3 Hyper-text Transfer Protocol (HTTP)	16
3.4 JavaScript Object Notation (JSON)	18
3.5 YAML Ain't Markup Language	19
3.6 CORS	19
4 Designing and documenting APIs	23
4.1 Introduction to APIs	23
4.2 OpenAPI Specification Overview	24
4.3 Designing REST API	24
5 Backend programming with Go	31
5.1 Introduction to Go	31
5.2 Concurrency	44
6 Web Front-end Programming with Vue.js	47
6.1 HTML	47
6.2 CSS	49
6.3 Javascript	52
6.4 Vue.js	55
7 Containers with Docker	69
7.1 Introduction to Linux Containers and Docker	69
7.2 Running Your First Container	71

Chapter 1

Introduction

The **Web and Software Architecture (WaSA)** course offers a comprehensive introduction to the world of full-stack web development. It is designed to equip students with essential skills to design and implement modern web applications using contemporary frameworks, languages, and tools.

The WaSA course has several fundamental goals, each aimed at building a strong foundation in web development:

1. *Standard Tools and Frameworks:* Students will gain hands-on experience with key industry tools, such as git for version control and Vue.js, a popular JavaScript framework for building user interfaces.
2. *Web Markup Languages:* Students will learn the core languages of web development — HTML for structure and CSS for styling — necessary for creating well-formed web pages.
3. *Powerful Programming Languages:* The course delves into two primary programming languages: JavaScript, essential for frontend development and Go used predominantly for backend development.
4. *API Design and Implementation:* Students will learn how to design and implement an Application Programming Interface (API), a crucial component for creating scalable web applications that can communicate between different systems.
5. *Client-Server Architecture:* The course will guide students through the design and development of a complete client-server web application, preparing them to handle both frontend and backend responsibilities.
6. *Software Containers and Deployment:* Finally, students will learn to package their applications into containers using Docker and deploy them in scalable environments, such as Kubernetes.

1.1 Web apps

Web applications, commonly known as web apps, are a significant part of modern computing, forming the backbone of how users interact with content, services, and functionality on the internet. But before delving into the specifics of web apps, it's important to understand what an application generally represents.

In the simplest terms, an application is a computer program designed to perform specific tasks. Applications can take many forms and run on various platforms, including desktop computers, mobile devices, and web browsers. Let's explore the different types of applications to understand how web apps fit into the broader landscape of software development.

1. *Desktop Applications:* A desktop application is a software program that runs directly on the operating system of a desktop or laptop computer. It typically requires installation and may or may not need an internet connection to function.

2. *Mobile Applications*: Mobile apps are designed to run on mobile devices such as smartphones, tablets, and smartwatches, operating within mobile operating systems like iOS or Android. Like desktop apps, they need to be installed from an app store and may also require internet connectivity for some functionalities. Mobile apps can be further divided into: *Native Apps*, these are applications specifically written for a particular mobile operating system, using languages such as Swift (for iOS) or Java/Kotlin (for Android); *Hybrid Apps*, use web technologies and are compiled to run on various platforms. and, as a single codebase, can be adapted for multiple operating systems.
3. *Web Applications*: Web apps represent a distinct type of application that operates entirely within a web browser. Unlike desktop or mobile apps, web apps don't require installation. They can be accessed via URLs and need a stable internet connection to function. Web apps differ from traditional websites in their interactivity and complexity. Each user interaction in a traditional web app leads to the loading of a new webpage, which contains an updated user interface (UI) based on the interaction.
4. *Single Page Applications (SPA)*: Single Page Applications take web app development to a new level by dynamically rewriting the content on the same page rather than loading new pages from the server with each interaction. The user interface remains intact, while only the relevant data is updated.
5. *Progressive Web Applications (PWA)*: Progressive Web Apps combine the best aspects of both web and mobile apps. A PWA is a web application that can be added to a device's home screen, allowing users to access it offline or in low-quality network conditions.

A web application is essentially software that runs within a web browser. Unlike traditional software applications, web apps split their operations between two distinct environments: the client-side (front-end) and the server-side (back-end). The front-end of a web app is the part of the application that users interact with directly through their browser. It's often referred to as the presentation layer. The front-end is built using markup languages like HTML for structure, CSS for styling, and JavaScript for interactivity. Additional techniques, such as AJAX (Asynchronous JavaScript and XML), are used to update parts of a web page without reloading the entire page. The back-end is where the core functionality of a web app resides, often referred to as the data access layer. The back-end can be built using various languages, ranging from scripting languages like PHP and Python to compiled languages like Java, C#, or Go. It is responsible for managing the application's logic, handling user requests, interacting with databases, and returning the necessary data to the front-end for display. The back-end also deals with Application Programming Interfaces (APIs), which act as bridges between the client and server, facilitating communication and data exchange. An API provides the necessary endpoints that allow the client to request specific services or data from the server.

Chapter 2

Version Control with Git

2.1 Version Control System

Version control systems (VCS) are tools designed to help manage changes to documents, code, and other complex projects over time. This is particularly important in projects where multiple collaborators are working simultaneously on the same files, ensuring consistency and traceability. In the absence of such systems, keeping track of different versions often leads to confusion and disorganization, where it becomes difficult to track which version is current or accurate.

The need for version control becomes critical in scenarios such as:

- *Complex Projects:* Documents or codebases that evolve over time require systematic tracking of changes.
- *Collaboration:* Multiple developers or team members often work on the same files, potentially introducing conflicting changes.
- *Parallel Versions:* Projects may have multiple branches for testing, development, and stable releases running concurrently.
- *Reproducibility:* For debugging or tracking down issues, it's crucial to revert to earlier versions and understand the evolution of the project.

A VCS offers several vital features:

1. *Consistent Labeling:* Each revision is labeled consistently to allow for easy identification of different versions.
2. *Tracking Changes:* Differences between revisions are logged, ensuring a clear history of modifications.
3. *Metadata:* Information such as dates, authors, and commit messages are associated with each change.
4. *Revisions:* The ability to create point-in-time snapshots of the entire project is critical for ensuring consistency.
5. *Collaboration:* Multiple developers can work on the same codebase concurrently, with features such as branching and merging to avoid conflicts.

There are three primary types of VCS:

1. *Local Version Control System:* A local VCS stores changes on the user's computer. Although it provides basic version tracking, collaboration is difficult without additional tools (see Figure 2.1).
2. *Centralized Version Control System (CVCS):* Centralized systems, use a single central server to store all versions of a project. Developers check out a working copy from the central repository, make changes, and commit them back (see Figure 2.2).

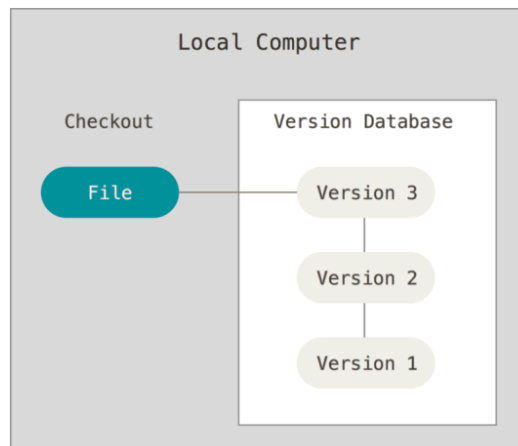


Figure 2.1: Local Version Control System

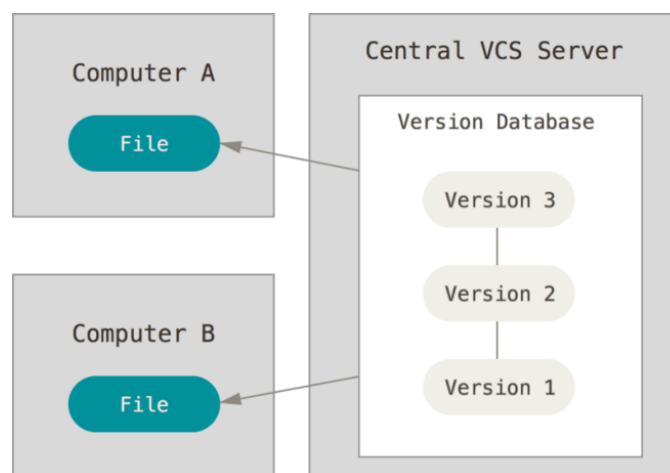


Figure 2.2: Centralized Version Control System

3. *Distributed Version Control System (DVCS)*: In DVCS, such as Git and Mercurial, each user has a full copy of the repository, including its history. This allows for full collaboration even without a central server, with users able to synchronize changes between repositories (see Figure 2.3).

Key Version Control Terminology Understanding some basic VCS terms is essential for working with any system:

Working Copy: The project files on your local machine that are tracked by the VCS. **Commit:** A snapshot of the repository at a specific point in time. Commits include changes, metadata (author, date), and often a commit message explaining the changes. **Branch:** A separate line of development. Branches allow for parallel work without affecting the main project. Each branch is essentially a sequence of commits. **Merge:** The process of combining changes from one or more branches into another. Merging helps integrate parallel work. **Tag:** A custom label applied to a specific commit. Tags are used to mark important milestones such as releases. **Repository:** The complete set of commits, branches, and tags for a project. It represents the entire history of changes made to the project. **Fork:** A personal copy of someone else's repository, typically used for developing independent changes that can later be merged back. **Pull/Merge Request:** A request to merge changes from one branch or fork into another, typically used in collaborative development workflows.

2.2 Git

Git is a distributed version control system (DVCS) created by Linus Torvalds in 2005 to manage the Linux kernel's source code. Git is designed to efficiently handle everything from small to very large

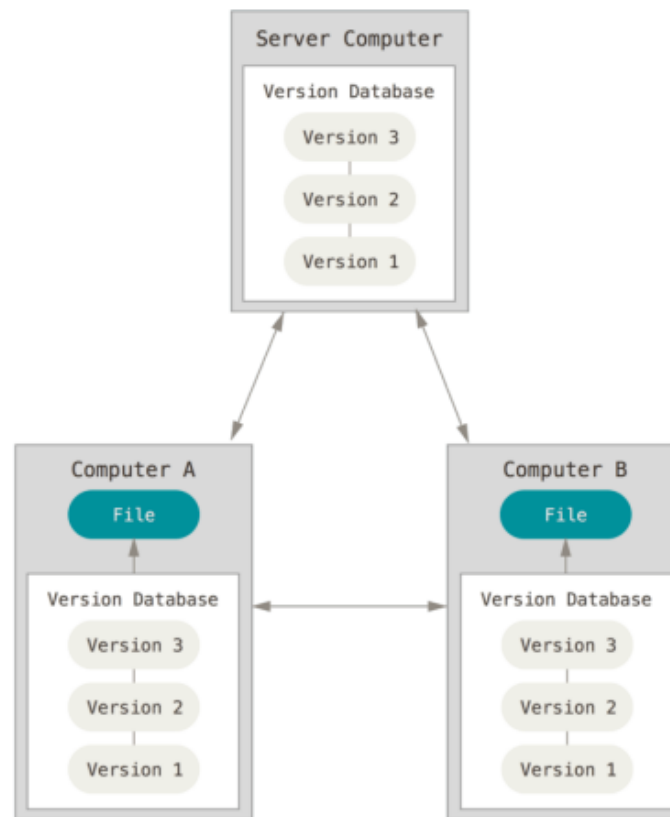


Figure 2.3: Distributed Version Control System

projects with speed and flexibility. It uses concepts like Directed Acyclic Graphs (DAG) and Merkle Trees for data integrity and fast operation.

Git Commit

A commit is a snapshot of the project at a given moment. It can include any subset of files, and each commit has one or more parent commits, except for the very first commit (which is an orphan). Commits require a meaningful message to explain the changes (see Table 2.1).

Component	Description
Commit message	A short description of the changes made.
Committer and date	The person who made the commit and when it was committed.
Author and author date	The original author of the change.
Tree	A hash representing all files in the commit.
Parent commits	One or more references to previous commits.
Commit ID	A SHA-1 hash of the commit's content.

Table 2.1: Structure of a Commit

Git Staging Area

The staging area is where files are added before committing. Only files in the staging area will be included in the next commit. This allows the developer to carefully curate changes to be committed.

Branching in Git

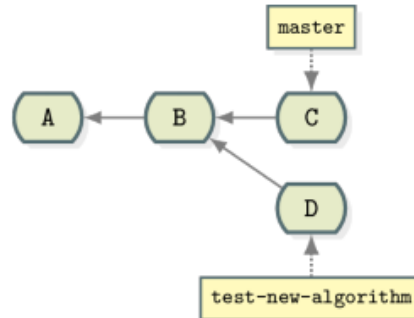
A branch is a line of development, essentially a sequence of commits. Branches allow for parallel development, such as creating a new feature or testing code without affecting the main project. When

a branch is ready to be incorporated back into the main project, a merge occurs.

The main branch (often called `main` or `master`) is the default branch where final code lives. Branches can be merged back into the main branch after development is completed.

Example 2.2.1 – Branching in Git

In this diagram, we see two branches of development in a Git repository: the master branch and a feature branch called `test-new-algorithm`.



Commit C is part of the master branch. This means that after commit B, a commit was made to the master branch. Commit D is part of the feature branch called `test-new-algorithm`, which diverged from commit B. This means that after commit B, a new branch was created to work on a different feature or experiment, which resulted in commit D. Eventually, these two branches (the master and feature branch) might need to be merged. The goal of merging would be to integrate the changes made in both the branches.

The HEAD

HEAD is a pointer to the current branch or commit you are working on. The files in your working directory reflect the state of the commit that HEAD points to. When HEAD points directly to a commit instead of a branch, it is referred to as a detached HEAD, which prevents committing new changes until you reattach it to a branch.

Tags in Git

Tags are labels used to mark specific commits, often for significant events like software releases. Tags help quickly find important points in a project's history. Although tags can be removed, it's generally best practice to avoid doing so.

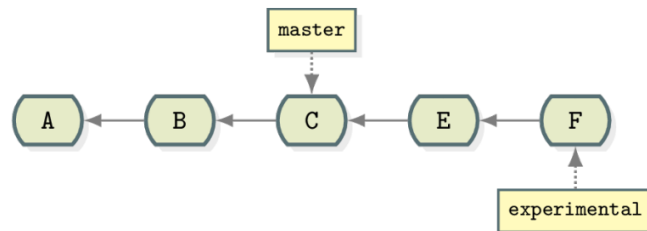
Merge Strategies

When merging branches, there are different strategies:

1. *Fast-forward merge*: Occurs when the branch being merged has no new commits since it branched off. The branch simply advances to include the new commits.
2. *Non-fast-forward merge*: Used when the two branches have diverged, requiring a merge commit to reconcile differences.
3. *Rebase*: An alternative to merging that replays commits from one branch onto another, resulting in a cleaner project history without merge commits.

Example 2.2.2 – Fast-Forward Merge in Git

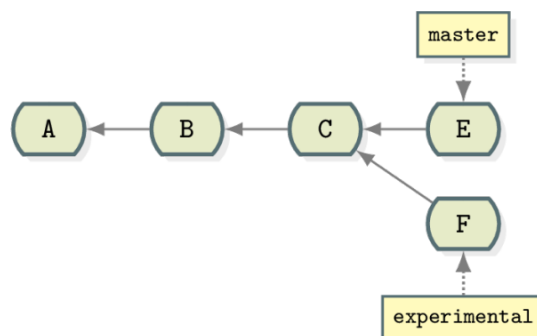
The master branch is currently pointing to commit C, while the experimental branch has advanced and contains commits E and F.



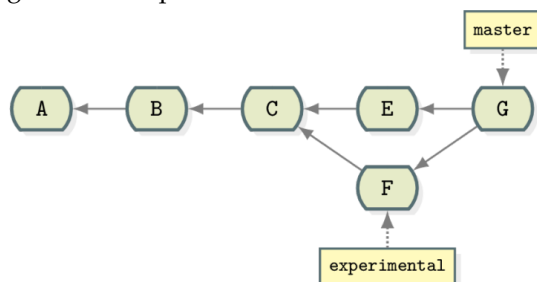
After performing a fast-forward merge, the master branch now points to commit F, which was the latest commit in the experimental branch. No new merge commit is created because fast-forward simply moves the pointer of the master branch to the latest commit of the merged branch (in this case, commit F). The history appears linear, as if the experimental changes were part of the original development line all along.

Example 2.2.3 – Non-Fast-Forward Merge in Git

In the first diagram, we observe two branches: master is pointing to commit E and experimental is pointing to commit F, which diverged from the master branch at commit C.

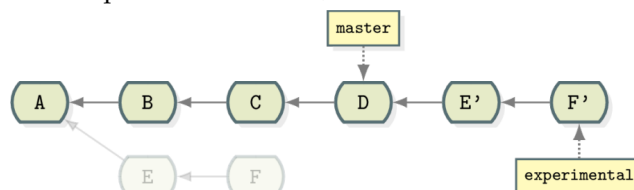


After the merge, a non-fast-forward merge is performed, and a new commit G is created, which contains the merged changes from both the master and experimental branches. The master branch is now updated to point to this new merge commit G. The experimental branch remains unchanged and still points to commit F.



Example 2.2.4 – Rebase Merge in Git

We see two branches: master is pointing to commit D and experimental has diverged from the master at commit B and points to commit F, with commit E as an intermediate step.



After the rebase, the experimental branch has been rebased onto the master branch: the commits E and F have been rewritten as new commits E' and F'. The experimental branch now appears as if it was developed directly on top of commit D (the latest commit on the

master branch). The old commits E and F are no longer referenced in the new history, effectively making them obsolete. It replays the changes from `experimental` on top of the `master` branch, ensuring that the history appears linear and avoids a merge conflict.

Merge Conflict

Merge conflicts arise when we merge branches, using non-fast-forward or rebase strategies, that modify the same part of a file in incompatible ways. Git will alert you to the conflict, and you must manually resolve it by choosing one of the following:

- *Edit the file manually*: Combine the changes as needed.
- *Keep "ours"*: Use your version of the file.
- *Keep "theirs"*: Use the other branch's version.

Remote Git Repositories

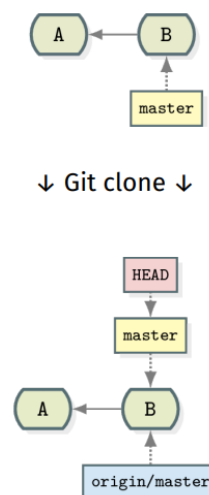
Git allows users to interact with remote repositories, which are hosted on remote servers. These repositories can store and share commits, branches, and tags. The most common remote repository is called `origin`, which is typically set as the default when a repository is cloned from a URL. A common workflow involves:

- *Cloning*: Cloning creates a local copy of a remote Git repository using a URL. This process downloads all the commits, branches, and tags into the local system. The remote URL from which the repository is cloned is assigned as the `origin` remote.
- *Pushing*: Once changes are committed locally, they can be sent to the remote repository using the `push` command. This uploads the local commits, branches, and tags to the remote.
- *Fetching and Pulling*: `Fetch` retrieves the latest changes from the remote repository, including commits, branches, and tags, but does not automatically merge them into your local branches. `Pull` is a combination of `fetch` and `merge`. After fetching the latest changes from the remote, Git automatically merges them into the current branch.

Example 2.2.5 – Git clone

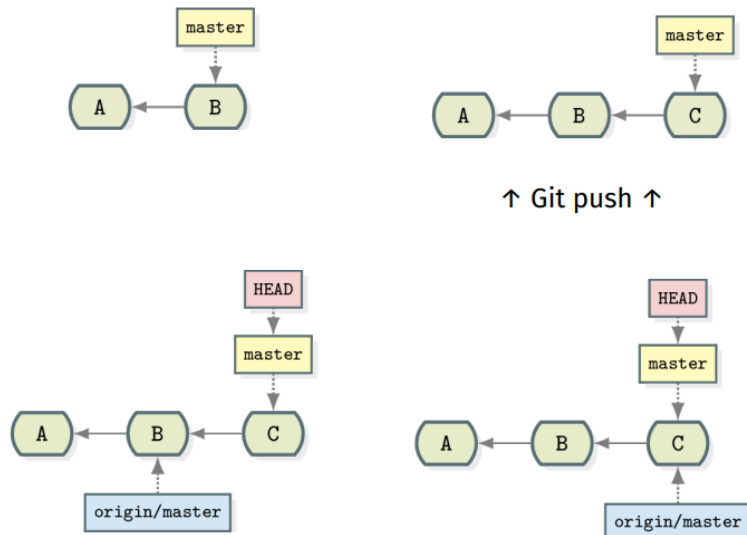
The diagram illustrates two states: the remote repository and the local repository after cloning. At the top, the remote repository is represented with two commits: Commit A and Commit B. The `master` branch pointer is positioned at commit B, indicating that this is the latest commit on the `main` branch.

Once the cloning operation is complete, we observe the local setup: The commits A and B are copied to the local repository. A local branch named `master` is created, pointing to commit B, similar to the remote. The `HEAD` points to the local `master` branch. The local repository also tracks the remote branch with the pointer `origin/master`, which is a reference to the latest commit on the remote repository's `master` branch. This tracking allows the local repository to know where the remote branch is in relation to the local copy.



Example 2.2.6 – Git push

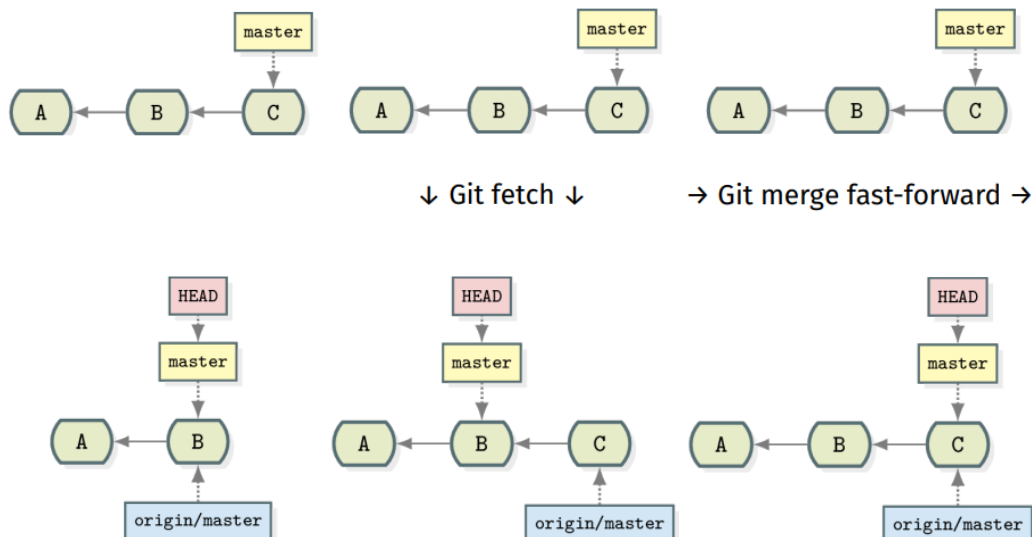
Before the push, the remote repository initially has two commits: A and B and the `master` branch pointer is positioned at commit B, indicating that it is the latest commit in the repository. The local repository contains the same two commits as the remote repository.



A new commit, C, has been made in the local repository, but it has not yet been pushed to the remote. The HEAD is pointing to the local master branch, which now includes commit C. The origin/master pointer still points to commit B, because that is where the remote repository is.

The `git push` command sends the new commit from the local repository to the remote repository. This action updates the remote repository with the local changes. After the push, the remote repository now contains commit C in addition to A and B. The master branch in the remote repository is updated to point to commit C, reflecting the new changes pushed from the local repository. The local repository remains the same after the push, with the HEAD still pointing to commit C. The origin/master reference is updated to commit C, indicating that the local and remote repositories are now synchronized, both containing commits A, B, and C.

Example 2.2.7 – Git fetch



Before the fetch, the remote repository has commits A, B, and C on the master branch and the local repository only has commits A and B, and the master branch points to B. There's also an outdated reference to the remote branch, origin/master, still pointing to commit B. At this point, the local repository is not up to date with the latest commit (C) from the remote master branch.

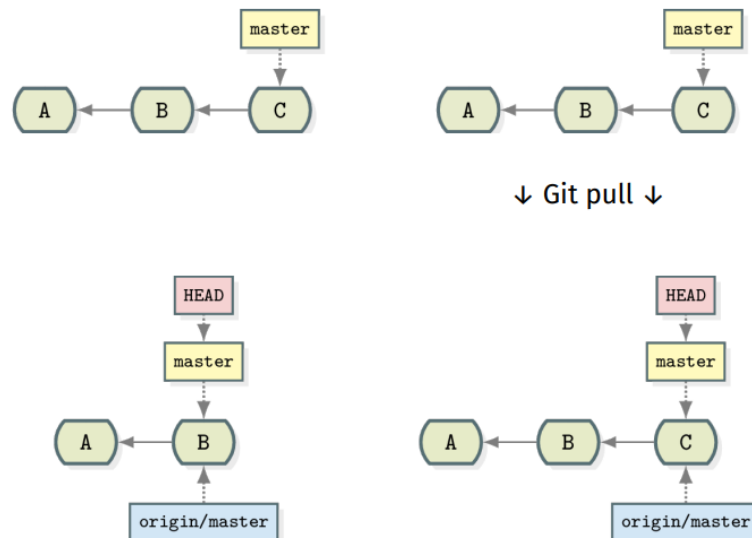
After running `git fetch`, the local repository retrieves the latest state of the remote branch. No changes have occurred in the remote repository; it still has commits A, B, and C on the master branch. The local origin/master reference is updated to point to commit C, reflecting

the state of the remote repository. However, the local `master` branch still points to B, indicating that the local working branch has not yet been merged with the new changes. At this stage, the local repository has retrieved the latest remote changes, but they have not been integrated into the local branch (`master`).

After running a fast-forward merge (`git merge --ff-only`), the local branch is updated. Again, no changes in the remote repository. The local `master` branch is now updated to commit C, aligning with `origin/master`. This is a fast-forward merge because the local branch was directly behind the remote branch, with no new changes of its own. As a result, the `master` branch simply advances forward to the latest commit (C) without creating any merge commit.

Example 2.2.8 – Git pull

Before `git pull`, the remote `master` branch has commits A, B, and C and the local repository is behind the remote and only has commits A and B. The `HEAD` and `master` branch in the local repository point to commit B, while the remote-tracking branch (`origin/master`) also points to B. This means the local repository hasn't fetched or pulled the latest changes from the remote, which includes commit C. At this point, the local branch needs to be updated to include the changes made in the remote repository.



After running `git pull`, no changes occur in the remote repository, which still has commits A, B, and C on the `master` branch. The local repository now has commit C after the pull operation. Both the `master` branch and the remote-tracking branch (`origin/master`) point to commit C, and the `HEAD` pointer is updated accordingly.

Git Forges

A Git forge is an online platform designed to facilitate collaboration on Git-based projects. Popular Git forges include GitHub, GitLab, and Bitbucket. These platforms often offer additional features like:

- *Bug tracking*: Managing issues related to the project.
- *Documentation wikis*: Collaborative spaces for writing documentation.
- *Forks and pull requests*: Enabling developers to copy a repository, make changes, and propose merging those changes back into the original project.
- *CI/CD integration*: Automating tests and deployments.

Chapter 3

Web protocols

3.1 REpresentational State Transfer (REST)

In modern web development, REST (REpresentational State Transfer) plays a pivotal role in designing distributed systems, particularly in the context of web services. REST defines a set of architectural principles that govern how networked resources should be transferred between clients and servers.

At its core, REST is an architectural style used for distributed hypermedia systems. This architecture facilitates communication between various components, such as a client and a server, by transferring representations of resources. The term "representational" refers to the fact that REST doesn't directly expose resources but offers a current representation of them through a medium like JSON, XML, or HTML. A resource in REST refers to any identifiable information or entity that can be accessed or manipulated. This could be anything from a document, image, or service, to a real-world object like a person or a collection of data. The critical feature of a resource is its ability to vary over time. For instance, a resource might represent the latest version of a software program, which would naturally evolve as new updates are made.

When interacting with a resource in a RESTful system, what is transferred between components is not the resource itself but its representation. The representation of a resource encapsulates its current state, which may include metadata and a structured format such as JSON or XML. This data format is known as the media type.

3.2 Uniform Resource Identifier (URI)

In REST, each resource is uniquely identified by a Uniform Resource Identifier (URI), which is a string of characters that points to a resource's location. A URI can represent either a logical or physical resource and typically takes the form of a URL, such as: `http://example.com/users`. This URI addresses the "users" resource on a given server, allowing clients to access or manipulate the resource by sending HTTP requests to this address.

Notation 3.2.1 – URI

When constructing URIs, REST adheres to several conventions to maintain simplicity and readability: Forward slashes (/) express hierarchy within the URI structure. However, trailing slashes should only be used when the resource is not a "leaf" node (i.e., it is not a final, single resource). Use hyphens (-) rather than underscores (_) to separate words in a URI, as hyphens are more readable. Stick to lowercase letters for consistency. Avoid file extensions in URIs (such as .html or .json), as the media type is specified in the HTTP headers. Use query components to filter or refine the results. For instance, to access only resources from a specific region: `http://example.com/managed-devices/?region=USA`.

URIs serve the purpose of identifying resources, but the actions performed on these resources are not specified within the URI. Instead, REST utilizes HTTP methods to interact with resources. Common HTTP methods include:

- *GET*: Retrieve the representation of a resource.

Example: GET `http://example.com/managed-devices/{id}`.

- **PUT:** Update or create a new resource at the specified URI.

Example: PUT `http://example.com/managed-devices/{id}`.

- **DELETE:** Remove a resource.

Example: DELETE `http://example.com/managed-devices/{id}`.

REST operates on five fundamental constraints that ensure scalability, simplicity, and performance:

1. *Client-Server:* The client-server model enforces a clear separation of concerns: the client focuses on the user interface, while the server manages data storage and business logic. This separation enhances scalability and portability, making it easier to update the system without disrupting users.
2. *Stateless:* Each client request must be self-contained. The server does not maintain any session state between requests, meaning that every request carries with it all the information needed to process it. This simplifies server design and allows for easy scalability, as servers don't need to track individual clients.
3. *Cacheable:* REST supports cacheable responses, meaning that clients can store resource representations for later use, thus reducing the need for repeated server requests. The duration for which a resource can be cached is typically specified in the response headers.
4. *Uniform Interface:* REST employs a uniform interface to standardize the interactions between clients and servers. This ensures that implementations are decoupled from the services they provide, promoting interoperability across systems. There are four main constraints for the uniform interface: *Identification of Resources:* Resources must be uniquely identifiable. *Manipulation of Resources Through Representations:* Clients interact with resources via representations, such as JSON or XML. *Self-descriptive Messages:* Each message exchanged between components should contain enough information to describe how it is processed. *Hypermedia as the Engine of Application State (HATEOAS):* The client only needs the initial URI to interact with the system.
5. *Layered System:* In a layered system, different components handle different aspects of communication. For example, requests may pass through a proxy, gateway, or intermediary before reaching the origin server. These intermediary components forward requests and responses, potentially translating them or adding extra functionality such as load balancing. Each component interacts only with the immediate layer next to it, enhancing modularity and security.

3.3 Hyper-text Transfer Protocol (HTTP)

The HyperText Transfer Protocol (HTTP) is a cornerstone of the modern web, governing how information is exchanged over the internet. Initially invented by Tim Berners-Lee between 1989 and 1991 while at CERN, it has evolved to become an essential part of the internet's application layer protocol suite. A more recent version, HTTP/3, was published in 2022, and alongside it, HTTPS (the secure version of HTTP) offers enhanced security for web communications.

Client-Server architecture

At the heart of HTTP is the client-server model. The client, or user agent (UA), can be any application, such as a web browser, mobile app, or even smart appliances, that initiates requests. On the server side, the origin server (O) is responsible for responding to these requests, often providing authoritative data such as website content, video streams, or sensor readings.

When a user interacts with a web service, the client sends a request to the server, and the server responds accordingly. This exchange of information is the foundation of how we experience the web today.

A typical HTTP interaction follows a simple request-response model. For example, a client might send a request like this:

```
GET /hello.txt HTTP/1.1
User-Agent: curl/7.64.1
Host: www.example.com
Accept-Language: en, it
```

This is a basic GET request, asking the server for the file `hello.txt`. The server's response could be:

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Content-Length: 51
Content-Type: text/plain
```

Hello World! My content includes a trailing CRLF.

This response provides a status code of 200 OK, signaling a successful request, and includes details like the file's modification date and its content, "Hello World!".

HTTP interactions can also involve intermediaries such as proxies, gateways, or tunnels. These intermediaries can process or forward the requests and responses between clients and servers. In some cases, caches store previous responses to reduce the need for repeated requests to the server. For example, a cache can hold a stored response, which the client can reuse if the server has declared the resource as cacheable. However, tunnels, unlike proxies, do not store responses.

HTTP Methods

HTTP defines several methods that indicate the desired action to be performed on the resource identified by the request URI. These include:

- GET: Requests a representation of the resource. It is safe and cacheable.
- POST: Submits data to be processed to the resource, creating or modifying its subordinates.
- PUT: Creates or replaces a resource at the URI provided.
- DELETE: Removes the resource specified.
- HEAD: Like GET but without the response body, used to retrieve headers.
- CONNECT: Establishes a tunnel to a server, typically used for HTTPS.
- OPTIONS: Describes communication options for the resource.
- TRACE: Performs a message loop-back test along the request path.
- PATCH: Partially modifies a resource.

Each method has unique properties. For instance, GET and HEAD are considered safe because they do not modify resources, while PUT and DELETE are idempotent, meaning repeated requests yield the same outcome. POST, on the other hand, is neither safe nor idempotent, as multiple identical requests can result in different outcomes (e.g., creating multiple resources).

Some methods, like GET, HEAD, and under certain conditions POST, are also cacheable. This means that responses to these requests can be stored and reused by caches without needing to send another request to the origin server, potentially improving performance. However, cacheability often depends on specific conditions such as the presence of cache-control headers or expiration settings.

When a server responds to a request, it sends back a status code to indicate the result of the request. These codes are divided into five categories:

1. *1xx Informational*: The request was received and is being processed.
2. *2xx Successful*: The request was successfully processed. For example, 200 OK indicates success, while 201 Created means a new resource was created.

3. *3xx Redirection*: Further actions are needed to complete the request. A common example is 301 Moved Permanently, where the resource has been moved to a new URI.
4. *4xx Client Error*: There was a problem with the request. For instance, 404 Not Found means the server could not find the requested resource, and 401 Unauthorized requires authentication.
5. *5xx Server Error*: The server encountered a problem while processing the request. 500 Internal Server Error is a general error code for unexpected conditions, while 503 Service Unavailable means the server is overloaded or down for maintenance.

3.4 JavaScript Object Notation (JSON)

JSON is a lightweight data-interchange format that has become the backbone of many web applications. It is simple, readable by humans, and easy for machines to parse and generate, making it a favorite for exchanging data via RESTful APIs and storing information. Although it is based on a subset of JavaScript, JSON is language-independent, making it versatile across various programming environments.

Consider the following example of a JSON object that represents a user's data:

```
{
  "user": {
    "firstName": "John",
    "lastName": "Smith",
    "age": 27
  }
}
```

In this format, the data is structured as key-value pairs. JSON only uses two core concepts: objects and arrays, making it easy to grasp.

A JSON object is an unordered collection of key-value pairs, where each key is a string, and values can be various data types like numbers, strings, booleans, arrays, or even other objects. For example:

```
{
  "WASA": {
    "name": "Web and Software Architecture",
    "semester": 1
  }
}
```

An array in JSON is an ordered list of values, where each item can be of any JSON data type. Arrays are enclosed in brackets [] and separated by commas. Here's an example:

```
{
  "wasaWeekdays": ["tuesday", "thursday"]
}
```

Let's take a deeper look at how a more complex JSON structure might look. It includes a number, string, boolean, null value, an array, and another object inside the array:

```
{
  "anObject": {
    "aNumber": 42,
    "aString": "This is a string",
    "aBoolean": true,
    "nothing": null,
    "anArray": [
      1,
      {"name": "value", "anotherName": 12},
      "something else"
    ]
  }
}
```

This shows the flexibility of JSON in handling a wide range of data types in a structured format.

3.5 YAML Ain't Markup Language

YAML is a human-readable data serialization format often used for configuration files, data storage, and data exchange. It is valued for its simplicity, as it relies on indentation rather than brackets or braces like JSON. YAML also supports comments, which is a feature missing from JSON. Here's an example:

```
user:
  firstName: John
  lastName: Smith
  age: 27
```

YAML uses a simple syntax where indentation defines structure. Keys are separated from values using colons, and lists (arrays) can be expressed using dashes -. Here's a similar structure to the JSON example:

```
anObject:
  aNumber: 42
  aString: This is a string
  aBoolean: true
  nothing: null
  anArray:
    - 1
    - anotherObject:
        someName: some value
        someOtherName: 1234
    - something else
```

YAML is also more forgiving when it comes to format: it allows for multiline strings, block literals, and supports both JSON-like inline objects and arrays.

One key advantage of YAML is that it is a superset of JSON. This means that any valid JSON document is also valid YAML. For instance, JSON's use of {} for objects and [] for arrays is acceptable in YAML as well:

```
anotherArray: [1, 2, 3]
anotherObject: {"city": "Rome", "country": "Italy"}
aStringWithColon: "COVID-19: procedure di accesso"
aNumericString: "0649911"
```

Additionally, YAML allows for more advanced features, such as multiline strings:

```
aLongString: |
  this string spans
  more lines
```

This shows how YAML combines readability with versatility, making it an ideal choice for configuration files or data exchange in human-readable formats.

3.6 CORS

SOP

The Same-Origin Policy, or SOP, is a security mechanism enforced by web browsers to restrict interactions between resources from different origins. It acts as a safeguard, ensuring that malicious scripts from untrusted sources cannot interact with sensitive data from another origin.

What Defines an Origin? An "origin" is defined as the combination of a URI's scheme (e.g., http or https), hostname, and port number. For example:

- http://bank.com and https://bank.com are different origins because of the scheme.
- http://bank.com:8080 and http://bank.com are different origins because of the port.

Imagine a scenario where a user visits a banking website (https://bank.com) that includes the following HTML:

```
<!doctype html>
<html>
<head>
  <!-- Head content -->
</head>
<body>
  <!-- Content -->
  <script src="/script.js"></script>
</body>
</html>
```

In this setup, the script (`/script.js`) is allowed to interact with the resources hosted on `https://bank.com`. However, if a malicious script from a different origin (`https://hackedpartner.com/evil-script.js`) were introduced into the page, SOP would restrict its access to sensitive data like cookies, HTTP authentication tokens, or IndexedDB storage.

SOP ensures that critical components of web applications are protected from cross-origin interactions. These include:

- Cookies
- HTTP Authentication
- IndexedDB
- Web Storage (e.g., `localStorage`, `sessionStorage`)
- DOM manipulation

By restricting unauthorized access to these components, SOP plays a vital role in maintaining web security.

While SOP is effective, it's not a complete solution. For example:

1. *Writes*: Cross-origin writes (e.g., sending a POST request) are typically allowed but may require additional safeguards like CORS.
2. *Embedding*: Resources like images or scripts can be embedded from other origins without restrictions.
3. *Reads*: Reading responses from cross-origin requests is generally denied.

A significant limitation of SOP is its inability to prevent Cross-Site Request Forgery (CSRF) attacks. These occur when malicious websites trick a browser into executing unauthorized actions on another site. To mitigate CSRF risks, tokens or headers are often used in conjunction with SOP.

CORS

CORS, or Cross-Origin Resource Sharing (an extension to SOP) is a security feature implemented in web browsers to manage how resources are shared between different origins. It ensures that a web page can only access data from another domain if the target domain explicitly allows it. This mechanism is critical for maintaining web security and preventing unauthorized data access.

An "origin" in this context is defined by the combination of a URI's scheme, hostname, and port number. For example:

- `http://www.example.com` and `https://www.example.com` are different origins because the schemes differ.
- `http://www.example.com:8080` and `http://www.example.com` are also different because of the port number.

CORS comes into play when a script from one origin attempts to access resources on another origin, a practice known as a cross-origin request.

Imagine a scenario where a user agent (typically a web browser) loads a web page from `http://www.origin1.com`. A script on this page then tries to fetch data from another domain, `http://www.origin2.com`.

The browser sends a request to `www.origin2.com`:

```
GET / HTTP/1.1
Host: www.origin2.com
```

The key question is whether `www.origin2.com` will allow a script originating from `www.origin1.com` to access its data. Without CORS, this request would be blocked due to the browser's same-origin policy.

To determine whether `www.origin2.com` allows access, the user agent includes an Origin header in the request:

```
Origin: http://www.origin1.com
```

This header informs the target server (`www.origin2.com`) of the origin making the request. If `www.origin2.com` permits sharing its resources with `www.origin1.com`, it responds with the following header:

```
Access-Control-Allow-Origin: http://www.origin1.com
```

Alternatively, it can use a wildcard (*) to allow access from any origin:

```
Access-Control-Allow-Origin: *
```

If `www.origin2.com` does not allow cross-origin access, it will return an error, blocking the script from retrieving the data.

Not all HTTP methods are considered "SAFE." Methods like DELETE are potentially destructive and require stricter handling. If `www.origin2.com` supports CORS but does not permit a specific operation, it can deny the request and avoid executing the operation.

If the target server (`www.origin2.com`) does not support CORS, there is a risk that non-SAFE operations (like DELETE) could be executed unintentionally. To mitigate this, browsers implement preflight requests for non-SAFE operations.

Before performing a potentially unsafe operation, the browser sends a preflight request to the target server. This request is an OPTIONS request, asking the server if the operation is allowed. For example:

```
OPTIONS / HTTP/1.1
Host: www.origin2.com
Origin: http://www.origin1.com
Access-Control-Request-Method: DELETE
```

The server responds with headers that specify whether the requested operation is permitted:

```
Access-Control-Allow-Origin: http://www.origin1.com
Access-Control-Allow-Methods: DELETE
```

If the server's response confirms that the operation is allowed, the browser proceeds with the actual request. If not, the operation is blocked, ensuring that potentially harmful actions are not executed.

Types of CORS requests

CORS defines two types of requests: simple and preflighted.

A request is considered simple if it meets the following criteria:

- It uses one of the following HTTP methods: GET, HEAD, or POST.
- Headers are limited to a predefined set, such as Accept, Accept-Language, or Content-Type (restricted to application/x-www-form-urlencoded, multipart/form-data, or text/plain).

For example, a browser sending a simple request includes the Origin header:

```
Origin: https://bank.com
```

The server can then respond with:

```
Access-Control-Allow-Origin: https://bank.com
```

This indicates that only `https://bank.com` is allowed to access the response.

A preflighted request is required for operations that do not meet the criteria for a simple request, such as:

- Using HTTP methods like PUT, DELETE, or custom headers.
- Accessing non-standard resources or credentials.

Before making the actual request, the browser sends an OPTIONS request to the server to verify permissions. For example:

```
OPTIONS /resource HTTP/1.1
Host: api.example.com
Origin: https://bank.com
Access-Control-Request-Method: DELETE
```

Server Response:

```
Access-Control-Allow-Origin: https://bank.com
Access-Control-Allow-Methods: DELETE
```

If the server allows the operation, the actual request is sent.

When authentication is required, CORS simplifies the process in single-page applications (SPAs). For example, the Authorization header can be used for authentication, eliminating the need for CSRF tokens. Since Authorization triggers a preflight request, the server can verify the request's legitimacy through CORS configuration.

Chapter 4

Designing and documenting APIs

4.1 Introduction to APIs

In the modern digital world, software systems need to communicate with each other. To make this communication possible, developers use APIs, which stand for Application Programming Interface. APIs define the allowed interactions between two software components. These components could range from two different systems or applications to a consumer and a service provider. The API specifies how requests should be made, the parameters needed, the format for sending data, and the expected return values.

Imagine APIs as contracts between the consumer (the one making the request) and the provider (the one delivering the service). Both parties must adhere to the rules defined by the API to ensure smooth communication. Without this explicit definition, it would be challenging to manage interactions between different parts of a system.

One of the key advantages of using APIs is that they provide an explicit interface. This means that APIs clearly define the methods for interacting with the software, ensuring there's no ambiguity. Another significant benefit is information hiding. With an API, a consumer does not need to know how the provider works internally. This abstraction is valuable because it allows developers to focus on their specific task without worrying about the complexities of the service they are interacting with. APIs also establish an unbreakable contract between the consumer and the provider. Once defined, the provider is committed to maintaining the API so as not to break compatibility for existing consumers. This ensures reliability and consistency.

APIs can be categorized based on where and how they operate. For instance, there are local APIs that might be used within a single programming language or within a specific system. Examples include APIs in operating systems, software libraries, and hardware interfaces. These APIs allow different components of a single system to interact seamlessly.

On the other hand, remote APIs are used when different systems or applications need to communicate over a network. A common example of this is a web API, where a client sends a request to a remote server, and the server sends back a response, often in formats like JSON or XML. Web APIs are at the heart of modern web services, making remote systems easily accessible to developers.

APIs can also be categorized as public or private. A public API is one that is available for public use, such as the APIs provided by Google, Facebook, or Twitter. These APIs allow developers around the world to integrate their services into different applications. However, even public APIs can have restrictions. For instance, access might require the use of API tokens, which act as authentication keys to ensure that only authorized users can interact with the API.

On the other hand, private APIs are meant for internal use within an organization. These APIs are not available to the public and are typically used for internal communication between different systems or components of a company's software infrastructure.

An important aspect of APIs is interface stability. If an API changes, it can break the applications that depend on it. For this reason, unstable or experimental parts of an API might be marked as "beta" or "deprecated," indicating that they might change or be removed in future versions. Developers must be cautious when using such features to avoid compatibility issues.

APIs are defined through either documentation or description files. Documentation typically provides a free-text explanation of how the API works, the types of requests that can be made, and examples of data formats. However, relying solely on documentation has its drawbacks. Documen-

tation can become outdated, be unclear, or fail to cover all the edge cases of an API.

4.2 OpenAPI Specification Overview

To address these shortcomings, many APIs also come with a machine-readable specification, such as the OpenAPI Specification (OAS). This is an industry-standard format for describing HTTP-based APIs. OAS is human- and machine-readable, making it easier to generate code, automatically validate API requests, and create mock servers for testing purposes.

The OpenAPI Specification is a popular vendor-neutral format for describing APIs. This format is widely adopted and provides a standard way to document HTTP-based APIs. Let's look at an example of an OpenAPI description file written in YAML:

```
openapi: 3.0.0
info:
  title: An example OpenAPI document
  description: This API allows writing down marks on a Tic Tac Toe board
    and requesting the state of the board or of individual squares.
  version: 0.0.1
paths: {} # No endpoints defined yet
```

This simple description provides a title, a brief explanation of the API's purpose, and a version number. The paths section will eventually contain the endpoints available in the API, but it's empty in this example.

4.3 Designing REST API

Hi-Lo game

We explore the development of a web service for a simple game known as the Hi-Lo Game. The game is straightforward: the computer thinks of a secret number between 1 and 100, and the player has up to 10 attempts to guess the number. After each guess, the system will respond with one of three indications: too high, too low, or correct.

Before delving into the technical design, it's important to understand the service requirements. The Hi-Lo game service should provide the following functionalities:

1. Start a new game.
2. Accept a guess from the player, up to 10 guesses, and return whether the guess is too high, too low, or correct.
3. Reset the game by generating a new secret number.
4. Return a list of guesses for the current game, along with the results of each guess.
5. Provide a list of all games played, along with their outcomes (win/lose) and the number of guesses made.

From these requirements, it's evident that we need a system that can not only manage individual games but also maintain the history of all games. This immediately suggests the need for a well-structured API that can handle these operations efficiently.

To structure the API, we start by identifying the core resources. The key resource here is the game. Each game is essentially a distinct entity, and we can organize these entities using the following structure:

- /games: A collection representing all games. This will handle requests to create new games and retrieve the list of past games, including their outcomes.
- /games/{id}: Each individual game will be treated as a separate resource, allowing us to handle operations like guessing, resetting, and retrieving the status of a specific game.

This resource structure adheres to the principles of REST, where each resource is identified by a URI and manipulated using standard HTTP methods such as GET, POST, and PUT.

To formalize the design, we will use the OpenAPI specification, which allows us to define our API in a structured, machine-readable format. For this, we use the Swagger Editor, a widely used tool for creating and validating OpenAPI documents. Let's begin by defining the basic metadata for the Hi-Lo Game API:

```
openapi: 3.0.0
info:
  title: Hi-Lo Game
  description: This API allows playing the Hi-Lo game and retrieving the
    state of a game or the history of all games.
  version: 0.0.1
paths:
  /games:
    # This section will define the methods for interacting with the games
    collection
  /games/{id}:
    # This section will define the methods for interacting with
    individual games
```

This basic structure sets up the API's foundation, including the overall description and the paths for accessing the games collection and individual game instances.

The /games path will manage the collection of all games. The key operations here are:

1. POST: To start a new game.
2. GET: To retrieve a list of all games, including their results.

The OpenAPI definition for these methods would look something like this:

```
paths:
  /games:
    post:
      summary: start a new game
      description: Start a new game generating the secret number and
        return the created game id
      responses:
        "200":
          description: new game created
          content:
            application/json:
              schema:
                type: integer
    get:
      summary: list all games
      description: Obtain the list of all games, with the final result (
        win/lose) and the number of guesses
      responses:
        "200":
          description: An array of objects, each one containing the game
            id, the final outcome, and the number of guesses
          content:
            application/json:
              schema:
                type: array
                items:
                  type: object
                  properties:
                    id:
                      type: integer
```

```

outcome:
  type: string
enum:
  - win
  - lose
guesses:
  type: integer

```

This defines two endpoints. The POST method allows a user to start a new game, returning a game ID that will be used in future requests. The GET method retrieves a list of all games, with each game represented by its ID, result (win or lose), and the number of guesses made.

The next step is to define the operations for individual games. The {id} in the path represents the ID of a specific game. Here are the methods we will define:

1. POST: To make a guess in the game.
2. GET: To retrieve the state of a game, including the list of guesses made.
3. PUT: To reset the game, generating a new secret number.

These methods can be specified in OpenAPI as follows:

```

/games/{id}:
  parameters:
    - name: id
      in: path
      required: true
      description: this is the game id
  schema:
    type: integer
    description: e.g., /games/1234
  put:
    summary: start or reset a game
    description: Start a game with given id, generating the secret
      number; or reset an existing game, re-generating the secret
      number and zeroing the guesses counter
    responses:
      "200":
        description: An existing game is reset
      "201":
        description: A new game is created
  post:
    summary: make a guess
    description: Try to guess the secret number; return hi, lo, or
      correct, plus the guess count.
    parameters:
      - name: guess
        in: query
        required: true
        description: the guess
    schema:
      type: integer
      minimum: 1
      maximum: 100
    responses:
      "200":
        description: Guess accepted, the result is in the the content
        content:
          application/json:
            schema:
              type: object

```

```

    properties:
      guess-count:
        type: integer
        minimum: 1
        maximum: 10
      guess-outcome:
        type: string
        enum:
          - hi
          - lo
          - correct
  "403":
    description: game over
    content:
      application/json:
        schema:
          type: object
          properties:
            id:
              type: integer
            outcome:
              type: string
              enum:
                - win
                - lose
            guesses:
              type: integer
  "404":
    description: game not found
get:
  summary: list guesses
  description: Return a list of all guesses in reverse chronological
    order, including the responses received (hi/lo/correct)
  responses:
    "200":
      description: Request accepted, all guesses listed in content
      content:
        application/json:
          schema:
            type: array
            items:
              type: object
              properties:
                guess-count:
                  type: integer
                  minimum: 1
                  maximum: 10
                guess-outcome:
                  type: string
                  enum:
                    - hi
                    - lo
                    - correct

```

The POST method takes a guess from the user, compares it to the secret number, and returns whether the guess is too high, too low, or correct. The GET method returns the history of guesses made in the game. The PUT method allows the game to be reset, generating a new secret number for the player to guess.

Reusing OpenAPI descriptions

Reusing objects in OpenAPI allows developers to reduce code duplication, which in turn minimizes maintenance efforts and the potential for errors. When defining an API, the same data structures or responses may be used in multiple places. Instead of redefining these objects each time, OpenAPI allows developers to create components, which are reusable descriptions that can be referenced wherever needed.

At the heart of reusability in OpenAPI is the `components` object. This object can hold reusable definitions such as parameters, responses, and schemas, which are often referred to by other parts of the API. Consider the example of the Hi-Lo game.

```
components:
  schemas:
    guess-result:
      type: object
      properties:
        guess-count:
          type: integer
          minimum: 1
          maximum: 10
        guess-outcome:
          type: string
          enum:
            - hi
            - lo
            - correct
    final-result:
      type: object
      properties:
        id:
          type: integer
        outcome:
          type: string
          enum:
            - win
            - lose
        guesses:
          type: integer
```

Now, whenever a schema is needed, the API can reference its component.

```
get:
  summary: list all games
  description: Obtain the list of all games, with the final result (win/lose) and the number of guesses
  responses:
    "200":
      description: An array of objects, each one containing the game id, the final outcome, and the number of guesses
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: "#/components/schemas/final-result"

post:
  summary: make a guess
  description: Try to guess the secret number; return hi, lo, or correct, plus the guess count.
  parameters:
```

```
- name: guess
  in: query
  required: true
  description: the guess
  schema:
    type: integer
    minimum: 1
    maximum: 100
responses:
  "200":
    description: Guess accepted, the result is in the the content
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/guess-result"
  "403":
    description: game over
    content:
      application/json:
        schema:
          $ref: "#/components/schemas/final-result"
  "404":
    description: game not found
get:
  summary: list guesses
  description: Return a list of all guesses in reverse chronological order
    , including the responses received (hi/lo/correct)
  responses:
    "200":
      description: Request accepted, all guesses listed in content
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: "#/components/schemas/guess-result"
```


Chapter 5

Backend programming with Go

5.1 Introduction to Go

Go, often referred to as Golang, is a programming language developed by Google. It was designed to streamline the development of software with a focus on simplicity, efficiency, and ease of use. Due to these strengths, several major companies have adopted Go for their own systems. Some prominent examples include Google, Dropbox, and Uber, which rely on Go for tasks that require highly scalable and performant code.

The simplicity of Go, along with its powerful concurrency features, make it especially useful for applications in web development and large-scale systems architecture.

One of the benefits of learning Go is the availability of the Go Playground, a web-based platform (found at go.dev/play/) that allows users to run Go code without installing any software on their local machine. This environment is ideal for experimenting with Go syntax and understanding its core functionalities.

Packages

In Go, code is organized into packages. Each package serves as a collection of related functions and types, all residing in one or more files within a directory. Every file in Go starts by declaring its package, often with the keyword `package main` for standalone applications.

```
package main
//...
```

The `main` package is unique, as it signals to the Go compiler that this package contains an executable program.

Go encourages modular code by allowing programs to import additional packages. This is done with the `import` statement, which can bring in essential libraries, such as `fmt` for formatted I/O or `math/rand` for generating random numbers.

```
import (
    "fmt"
    "math/rand"
)

func main() {
    fmt.Println("My favorite number is", rand.Intn(10))
}
```

Standard and external library

Go comes with a comprehensive standard library that provides a wide range of packages for common tasks. The full list of standard packages can be explored at pkg.go.dev/std, offering extensive resources for handling basic programming needs.

In addition to the standard library, Go allows you to create custom packages within your project or use external packages developed by others. External packages are useful for adding specific functionality that is not covered by the standard library. These packages can be shared among projects, encouraging code reuse and collaboration.

To create a custom package within a project, you first need to initialize a module. Modules are groups of related packages that make up a Go project. You can initialize a module by navigating to your project directory and running:

```
go mod init some-module
```

This command generates two important files in the project directory: `go.mod` and `go.sum`. The `go.mod` file specifies the module's path and dependencies, while `go.sum` records the checksums of the dependencies.

Once a module is initialized, you can create sub-packages by adding subdirectories. Each subdirectory represents a package, which can be imported using its path. For instance, if you create a package in `some-module/package1`, it can be imported with the following syntax:

```
import "some-module/package1"
```

Let's look at an example structure of a Go project with custom packages:

```
main.go
go.mod
go.sum
package1/functions.go
package1/other-things.go
package1/subpackage/file.go
```

In this structure, `main.go` is the main executable file, while `package1` is a custom package with several files (`functions.go` and `other-things.go`). There is also a subpackage inside `package1`, demonstrating how packages can be nested within a project.

Inside `package1`, you might have a file called `functions.go` that contains functions and variables. The file would start by defining the package name, like so:

```
package package1

// Note: Use an uppercase letter to make functions public.
func Dummy() {}
```

The `Dummy` function is a simple example function, and by capitalizing its name, it becomes an exported function, meaning it can be accessed from other packages that import `package1`.

In `main.go`, you can use the `Dummy` function by importing `package1`:

```
package main

import "some-module/package1"

func main() {
    package1.Dummy()
}
```

Go makes it easy to use external packages by downloading them from online repositories. To include an external package, use the `go get` command followed by the package's path. For example:

```
go get gopkg.in/yaml.v2
```

This command downloads the `yaml.v2` package, which can then be imported into your code.

```
package main

import "gopkg.in/yaml.v2"

func main() {
    yaml.SomeFunction()
}
```

Once imported, the external package functions are available for use, just like standard or custom packages.

Hello, World!

A classic introduction to any programming language involves writing a simple "Hello, World!" program. Go uses the `fmt` package to handle output formatting.

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, world!")
}
```

This program demonstrates several foundational concepts: importing the `fmt` package, defining the `main` function, and using `fmt.Println` to output text. Notably, Go functions and variables must start with a capital letter to be publicly accessible within imported packages.

Types

Go supports several fundamental data types, including:

- `bool` for boolean values.
- `string` for text.
- Integer types: `int`, `int8`, `int16`, `int32`, `int64`, along with their unsigned versions like `uint` and `uint8`.
- Floating-point numbers: `float32` and `float64`.
- Complex numbers: `complex64` and `complex128`.
- Special types like `byte` (alias for `uint8`) and `rune` (alias for `int32`, representing Unicode code points).

Variables in Go can be declared with the `var` keyword, either outside or inside functions. Variables declared within a function are local to that function, while those declared outside are visible throughout the package.

```
package main
import "fmt"

// these are visible in the package
var b bool
var s string
var x, y float32

func main() {
    var i int // this is local in this function
    fmt.Println(i, b, s, x, y)
}
```

The Go syntax places the type after the variable name, which is different from many other programming languages.

In Go, variables can be initialized with a specific value, or they will automatically receive a "zero value" if not explicitly initialized:

- Numeric types are initialized to 0.
- Booleans are initialized to `false`.

- Strings are initialized to an empty string "".

```
var i int = 10
var b = true // infers the boolean type from the initializer
var x, y = 1.5, 2.5
```

Within functions, Go allows for short variable declarations using the `:=` syntax. This shorthand both declares and initializes the variable.

```
package main
import "fmt"

func main() {
    k := 3
    fmt.Println(k)
}
```

Here, `k` is initialized to 3 without explicitly stating its type.

Constants in Go are declared using the `const` keyword. They are similar to variables but immutable, meaning their value cannot be changed after declaration.

```
const k = 10
const zero = -273.15
```

Arrays and slices

An array in Go is a fixed-length sequence of elements of a specific type. Arrays are declared with a specified size, and each element is initialized to its zero value by default.

```
package main
import "fmt"

func main() {
    var v [10]int
    odds := [5]int{1, 3, 5, 7, 9}
    i := 4
    v[0] = odds[i]
    fmt.Println(v)
}
```

In this example, `v` is an integer array of size 10, while `odds` is a fixed array initialized with specific odd numbers.

Slices in Go are more flexible than arrays. They represent a segment of an array and can be resized as needed. A slice is not an independent data structure but rather a view of an underlying array.

```
package main
import "fmt"

func main() {
    vowels := [5]string{"a", "e", "i", "o", "u"}
    var s []string = vowels[1:3] // ["e", "i"]
    fmt.Println(s)
}
```

Slices have a length, which is the number of elements in the slice, and a capacity, which is the number of elements from the slice's starting point to the end of the array.

Maps

Maps in Go are unordered collections of key-value pairs, where each key is unique. They are initialized using the `make` function or directly through literals.

```
package main
import "fmt"

func main() {
    temperatures := map[string]int{
        "Rome": 27,
        "London": 18,
        "San Francisco": 21,
    }
    fmt.Println(temperatures)
}
```

Pointers

Pointers in Go allow for the manipulation of memory addresses directly. A pointer is a variable that holds the address of another variable.

```
i := 42
p := &i // p now holds the address of i
fmt.Println(*p) // dereferences p to retrieve the value of i
*p = 21 // modifies i through p
```

Pointers enable efficient management of memory and are integral to many advanced programming techniques.

Custom types and structs

In Go, structs are a way to define complex data structures by grouping together variables under one name. A struct is similar to an object in other programming languages, but it has no methods or inheritance features. Instead, structs in Go are simply collections of fields, each with a specific data type. This makes structs powerful for organizing data in meaningful ways and is an essential feature when building scalable applications.

To define a struct in Go, you use the `type` keyword, followed by the name of the struct and the keyword `struct`. Each field is declared within curly braces and has a name and type.

Consider the example of a struct called `Vertex`, which contains two fields, `x` and `y`, both integers. This struct could represent a point in a 2D coordinate system:

```
package main
import "fmt"

type Vertex struct {
    x int
    y int
}

func main() {
    v := Vertex{1, 2}
    fmt.Println(v) // Output: {1 2}
}
```

In this example, the `Vertex` struct is initialized with the values 1 and 2 for `x` and `y` respectively. When printed, `fmt.Println(v)` outputs `1 2`, showing the values of `x` and `y` in the `Vertex` struct.

Struct fields can be accessed and modified using the dot `.` notation.

```
v.x = 4
fmt.Println(v) // Output: {4 2}
```

Here, we modified the `x` value of `v` to 4, while `y` remains 2. If we want to perform operations on the struct fields, such as calculating the squared sum of `x` and `y`, we can do the following:

```
m := v.x*v.x + v.y*v.y
fmt.Println(m) // Output: 20
```

This code calculates the expression $x^2 + y^2$, which equals 20 for the given values of `x` and `y`.

Pointers are integral to Go's memory management, and structs can be referenced using pointers. When you create a pointer to a struct, it allows you to modify the original struct data through the pointer. Here's how to use pointers with a `Vertex` struct:

```
p := &v
p.x = 5
fmt.Println(v) // Output: {5 2}
```

In this example, `p` is a pointer to `v`. By accessing `p.x`, we change `x` to 5 through the pointer, modifying the original struct `v` directly. When we print `v`, it shows `5 2`, indicating that the change made via the pointer has affected the original struct.

In Go, struct literals provide a convenient way to initialize structs with specific values for their fields. Struct literals can be written in several forms, as illustrated in the following examples:

```
var (
    v1 = Vertex{1, 2}           // Full initialization, v1 has type Vertex
    v2 = Vertex{x: 1}          // Partially initialized, y defaults to 0
    v3 = Vertex{}              // All fields are set to zero values, x:0 and y:0
    p  = &Vertex{1, 2}         // Pointer to a Vertex struct, type *Vertex
)
```

Here, `v1` is fully initialized with `x = 1` and `y = 2`, while `v2` only specifies `x = 1`, so `y` defaults to 0, Go's zero value for integers. `v3` initializes with both `x` and `y` as 0, using the default zero values for all fields. Finally, `p` is a pointer to a `Vertex` struct, which can be useful for efficiently managing memory and modifying the struct's fields directly.

Using struct literals is an efficient way to initialize and manage data within structs, especially in cases where many struct instances need to be created with similar default values.

Functions

In Go, functions are essential building blocks that allow developers to break down complex tasks into smaller, reusable sections of code. Functions enable code modularity, making programs easier to write, understand, and maintain. They also play a crucial role in defining a program's flow by encapsulating logic in organized units.

A function in Go is defined with the `func` keyword, followed by the function name, parameters, return type, and the body of the function. Parameters are defined by their name followed by their type, and unlike many other languages, Go places the type after the parameter name.

Consider the following example of a function called `add`, which takes two integers, `x` and `y`, as input parameters and returns their sum:

```
package main
import "fmt"

func add(x int, y int) int {
    return x + y
}

func main() {
    fmt.Println(add(42, 13))
}
```

In this program, the `add` function receives two integer arguments and returns an integer. When called with the values 42 and 13, it computes their sum and prints the result, which is 55.

One of Go's unique features is its support for multiple return values. This allows a function to return more than one result, which can be useful for conveying additional information about an operation. For instance, a division operation might need to return both the quotient and the remainder of the division.

The following example defines a function called `divide`, which takes two integers, `x` and `y`, and returns both the quotient and remainder:

```
package main
import "fmt"

func divide(x int, y int) (int, int) {
    return x / y, x % y
}

func main() {
    fmt.Println(divide(7, 3)) // Output: 2 1
}
```

The `divide` function divides `x` by `y` and returns two values: `x/y` (the quotient) and `x%y` (the remainder). When we call `divide(7, 3)`, it returns 2 and 1, representing the quotient and remainder respectively.

Go also offers named return values, which make function definitions more descriptive and often eliminate the need for explicit return values. In this setup, you can specify names for the return values in the function signature. Within the function body, these names act as regular variables, and you can assign values to them as you would any other variable.

```
package main
import "fmt"

func divide(x int, y int) (div int, mod int) {
    div = x / y
    mod = x % y
    return
}

func main() {
    fmt.Println(divide(7, 3)) // Output: 2 1
}
```

In this version of `divide`, we specify `div` and `mod` as named return values. Inside the function, we assign the quotient to `div` and the remainder to `mod`. Instead of writing `return div, mod`, we simply use `return`, which automatically returns the values of `div` and `mod` as specified in the function signature.

Control structures

In programming, control structures play a critical role in defining the flow of execution. Go provides various control structures to manage the logical flow of a program.

For loop

Go's only looping construct is the `for` loop, which can be adapted to different needs.

The first example illustrates a simple `for` loop where the program iterates from 0 to 5:

```
package main
import "fmt"
func main() {
    for i := 0; i <= 5; i++ {
        fmt.Println(i)
    }
}
```

In this snippet, the loop initializes `i` at 0, checks if `i` is less than or equal to 5, prints the value of `i`, and increments it by one on each iteration. As a result, it prints the numbers 0 to 5.

Go also allows you to use a for loop as a while loop. Here's an example:

```
sum := 1
for sum < 8 {
    sum += sum
}
fmt.Println(sum)
```

In this case, the loop continues as long as `sum` is less than 8, doubling `sum` with each iteration until the condition is no longer met.

It's possible to create an infinite loop by omitting the condition altogether:

```
for {
    // this will run indefinitely
}
```

If

The if statement in Go is straightforward and often used for decision-making. Here's an example that calculates the factorial of a number using recursion:

```
package main
import "fmt"

func fact(n int) int {
    if n <= 1 {
        return 1
    } else {
        return n * fact(n-1)
    }
}

func main() {
    fmt.Println(fact(5))
}
```

In this function, `fact` recursively calls itself until `n` is less than or equal to 1, at which point it returns 1.

Go allows a variable to be declared and used within the if statement itself, as demonstrated in the following example:

```
func pow(x, n, lim float64) float64 {
    if v := math.Pow(x, n); v < lim {
        return v
    } else if v > 2 * lim {
        return 99999 - v
    }
    return lim
}
```

Here, `v` is defined within the if statement, allowing it to be used within this block only. This function computes a power and compares it to a limit. If the computed value `v` is less than the limit, it returns `v`; otherwise, it performs additional checks.

Switch

The switch statement in Go is a powerful alternative to chained if-else statements. It evaluates each case from top to bottom, executing only the first matching case and exiting afterward.


```
package main
import ("fmt"; "time")

func main() {
    fmt.Println("When's Saturday?")
    today := time.Now().Weekday()
    switch time.Saturday {
    case today + 0:
        fmt.Println("Today.")
    case today + 1:
        fmt.Println("Tomorrow.")
    default:
        fmt.Println("Too far away.")
    }
}
```

This program checks the current day of the week and determines the proximity of the upcoming Saturday.

A switch can also be used without a condition, making it act like a sequence of if statements:

```
switch {
case t.Hour() < 12:
    fmt.Println("Good morning!")
case t.Hour() < 17:
    fmt.Println("Good afternoon.")
default:
    fmt.Println("Good evening.")
}
```

This form is useful when you need to evaluate multiple conditions based on different expressions.

Defer

The defer statement in Go is used to postpone the execution of a function until the surrounding function returns. Deferred calls are executed in Last-In-First-Out (LIFO) order.

```
func main() {
    defer fmt.Println("world")
    fmt.Println("hello")
}
```

When this code runs, it first prints hello, and then, upon reaching the end of the main function, it prints world. This behavior is particularly useful for closing resources or cleaning up after the main logic of the function has been executed.

A common use case for defer is to ensure that resources are released even if an error occurs:

```
file, err := os.Open("file.txt")
if err != nil {
    log.Fatal(err)
}
defer file.Close()
```

This code opens a file and defers its closure. Regardless of how the function exits, the file will be closed at the end, ensuring proper resource management.

Interfaces and methods

In Go, interfaces and methods are fundamental tools for designing modular and reusable code. They provide a way to define behavior that types can implement, allowing Go programs to achieve polymorphism—a concept where different types can be treated in a uniform manner if they share the same behaviors.

Interfaces in Go are collections of method signatures. If a type provides implementations for all the methods listed in an interface, it is said to satisfy the interface. This enables Go to implement “duck typing,” where a type’s compatibility with an interface depends on its methods, not on an explicit declaration.

Methods

Before diving into interfaces, it’s important to understand methods in Go, as they form the basis of interface implementation. Methods in Go are functions with a receiver argument, which is a special parameter that binds the function to a specific type, making it a method of that type. Structs, which are custom data types that group variables together, commonly serve as the receiver for methods in Go.

Consider a struct `Vertex`, which represents a point in a 2D coordinate system:

```
type Vertex struct {
    X int
    Y int
}

// Somewhere in the code
v := Vertex{1, 2}
fmt.Println(v)
```

We can add a method to `Vertex` to compare if two `Vertex` instances are equal:

```
func (v Vertex) Equal(other Vertex) bool {
    return v.X == other.X && v.Y == other.Y
}
```

In this example, `Equal` is a method on the `Vertex` type. It takes another `Vertex` as an argument and returns true if both `X` and `Y` values are the same.

In Go, methods can have either a value receiver or a pointer receiver. A pointer receiver allows the method to modify the original struct rather than a copy of it, which is particularly useful for memory efficiency with larger structs or when updates to the data are needed.

```
func (v *Vertex) Equal(other *Vertex) bool {
    return v.X == other.X && v.Y == other.Y
}
```

With `*Vertex` as the receiver, this `Equal` method operates on the pointer `v`, directly modifying or accessing the data of the original `Vertex`.

Go allows methods to be defined on any type, including user-defined types based on basic types. For instance, you could define custom types for latitude and longitude and add methods that validate their range:

```
type Latitude float64
func (lat *Latitude) IsValid() bool {
    return lat != nil && -90 <= *lat && *lat <= 90
}

type Longitude float64
func (lng *Longitude) IsValid() bool {
    return lng != nil && -180 <= *lng && *lng <= 180
}
```

In this example, `Latitude` and `Longitude` are types based on `float64`, with each having a method `IsValid` that checks if the values fall within acceptable ranges for geographic coordinates.

We can also create a `Point2D` struct that represents a coordinate pair and includes methods to validate the coordinate as a whole or to check if it’s at the origin:

```

type Point2D struct {
    Latitude Latitude
    Longitude Longitude
}

func (p Point2D) IsValid() bool {
    return p.Latitude.IsValid() && p.Longitude.IsValid()
}

func (p Point2D) IsZero() bool {
    return p.Latitude == 0 && p.Longitude == 0
}

```

Interfaces

An interface in Go is a collection of method signatures. If a type has methods that match all the signatures in an interface, that type implicitly satisfies the interface. This design enables Go's "duck typing" approach, where a type's compatibility with an interface is determined by the methods it implements.

A simple example is the `Stringer` interface in the `fmt` package, which defines a single method `String`:

```

type Stringer interface {
    String() string
}

```

Any type that implements the `String` method, returning a string, satisfies the `Stringer` interface.

To demonstrate how interfaces work, let's add the `String` method to the `Vertex` struct so that it satisfies the `Stringer` interface

```

type Vertex struct {
    X int
    Y int
}

func (v *Vertex) String() string {
    return fmt.Sprintf("(%d,%d)", v.X, v.Y)
}

// ...

func printSomething(s fmt.Stringer) {
    fmt.Println(s.String())
}

```

In this example, `Vertex` implements the `Stringer` interface by providing a `String` method. This method uses `fmt.Sprintf` to format `Vertex` values as `(X, Y)`. Now, whenever a `Vertex` instance is printed, the custom `String` method will be used, producing a nicely formatted string output. The `printSomething` function takes any type that satisfies the `Stringer` interface, allowing it to accept diverse types that have a `String` method.

Another commonly used interface in Go is the `Writer` interface from the `io` package, which defines a `Write` method:

```

// io/io.go:96
type Writer interface {
    Write(p []byte) (n int, err error)
}

```

The `Writer` interface is implemented by types that can write data. For example, `ResponseWriter` in the `net/http` package is a specialized version of `Writer` used to send HTTP responses. It includes additional methods such as `Header` and `WriteHeader`:

```
// net/http/server.go:95
type ResponseWriter interface {
    Header() Header
    Write([]byte) (int, error)
    WriteHeader(statusCode int)
}

func saveFile(w io.Writer, content []byte) {
    w.Write(content)
}
```

Here, `saveFile` accepts any type that satisfies the `Writer` interface, allowing the function to be used with a variety of types that handle data writing. The `saveFile` function doesn't need to know the details of the type—it only relies on the fact that it implements `Write`.

Errors handling

Error handling in Go is distinct from many other programming languages, as it does not rely on exceptions or try-catch blocks. Instead, errors in Go are treated as normal values that are returned by functions. This approach encourages developers to handle errors immediately and explicitly, improving code readability and reliability.

Errors are represented by the `error` type, which allows them to be returned alongside other values in a function. By convention, errors are usually returned as the last value in a function's return signature. For example:

```
func myFunction(a, b string) (string, int, error)
```

In Go, errors are meant to be handled immediately rather than postponed. This principle helps prevent unexpected failures and ensures that issues are dealt with as they arise. For example, when parsing a string to an integer, it's essential to check for errors right after the parsing attempt:

```
var someString string
value, err := strconv.ParseInt(someString, 10, 64)
if err != nil {
    // Handle the error immediately
}
```

There are several ways to handle errors within functions in Go. Let's explore the most common strategies.

Handling errors internally

One approach is to handle errors directly within the function where they occur. For example, the `parseOrZero` function attempts to parse a string to an integer but returns 0 if parsing fails:

```
func parseOrZero(someString string) int64 {
    value, err := strconv.ParseInt(someString, 10, 64)
    if err != nil {
        return 0
    }
    return value
}
```

Returning errors to the caller

In cases where the error needs to be managed by the calling code, Go encourages passing errors up the call stack. The `parseAndIncrement` function demonstrates this approach, incrementing a parsed integer if parsing succeeds, but returning an error if parsing fails:

```
func parseAndIncrement(strValue string) (int64, error) {
    value, err := strconv.ParseInt(strValue, 10, 64)
    if err != nil {
        return 0, fmt.Errorf("error parsing value: %w", err)
    }
    value++
    return value, nil
}
```

The choice between handling errors internally and passing them to the caller depends on the context. Sometimes, a function may handle certain errors while returning others to the caller.

Don't panic for error handling

While Go discourages the use of panic for general error handling, there are specific situations where it can be appropriate. The panic function abruptly stops the program and provides an error message, making it useful for "throwaway code" or cases where the program cannot continue safely.

```
func main() {
    var someString = "12345"
    value, err := strconv.ParseInt(someString, 10, 64)
    if err != nil {
        panic(err)
    }
}
```

In this case, panic halts the program if parsing fails. However, Go's best practices recommend limiting the use of panic to the main() function or situations where recovery is impossible.

Creating custom errors

Go allows developers to create custom error messages to provide more context when an error occurs. One way to do this is through errors.New, which generates a new error message.

Consider the example of parsing a latitude value and checking if it's within the valid range (-90 to 90 degrees):

```
func ParseLatitude(str string) (Latitude, error) {
    latitude, err := strconv.ParseFloat(str, 64)
    if err != nil {
        return 0, err
    } else if latitude < -90 || latitude > 90 {
        return 0, errors.New("value out of range")
    }
    return Latitude(latitude), nil
}
```

For more complex applications, it can be beneficial to declare error variables that describe specific conditions. By defining custom error variables, developers can check for particular error types in the calling code.

```
var ErrOutOfRange = errors.New("value out of range")

func ParseLatitude(str string) (Latitude, error) {
    latitude, err := strconv.ParseFloat(str, 64)
    if err != nil {
        return 0, err
    } else if latitude < -90 || latitude > 90 {
        return 0, ErrOutOfRange
    }
    return Latitude(latitude), nil
}
```

In the `main()` function, the custom error `ErrOutOfRange` can be checked using `errors.Is`, allowing the program to handle the error appropriately:

```
latitude, err := ParseLatitude(latitudeInString)
if errors.Is(err, ErrOutOfRange) {
    // Handle as invalid range
} else {
    // Handle other types of errors
}
```

For even more flexibility, Go allows developers to create structured error types by implementing the error interface. This can be useful when additional information, such as line numbers or detailed messages, needs to be included in the error.

The error interface in Go requires a single method, `Error()`, that returns a string representation of the error.

```
type MyError struct {
    LineNumber uint
    Message    string
}

func (e *MyError) Error() string {
    return fmt.Sprintf("Error in line %d: %s", e.LineNumber, e.Message)
}

func myFunction(str string) error {
    return &MyError{LineNumber: 2, Message: "Missing :"}
}
```

In this example, `MyError` is a custom error type with fields for a line number and a message. The `Error()` method formats these fields into a readable error string. When `myFunction` encounters an error, it returns a `MyError` instance with detailed information.

5.2 Concurrency

Concurrency is a programming approach that enables a program to manage multiple tasks at the same time. However, it's important to distinguish between concurrency and parallelism. As often quoted, "Concurrency is about dealing with lots of things at once, while parallelism is about doing lots of things at once." In other words, concurrency is more about structuring tasks efficiently, even if they aren't running simultaneously. Go was specifically designed with concurrency in mind, making it straightforward to implement concurrent programming through features such as goroutines and channels.

Go routines

In Go, the primary way to achieve concurrency is through goroutines. A goroutine is a lightweight thread that runs concurrently with other goroutines in the same address space, making it an efficient way to manage tasks.

```
func main() {
    go func() {
        var i = 0
        for i < 10 {
            fmt.Println(i)
            i++
        }
    }()

    var j = 0
```

```
    for j < 10 {  
        fmt.Println(j)  
        j++  
    }  
}
```

In this program, two loops are running: one inside a goroutine and the other in the main function. The `go` keyword in front of the anonymous function call makes it a goroutine, allowing it to execute concurrently with the main loop.

Channels

Channels in Go facilitate communication between goroutines by allowing them to send and receive messages. They are essentially typed conduits that synchronize goroutines.

```
func main() {  
    var channel = make(chan int)  
    go func() {  
        var i = 0  
        for i < 10 {  
            channel <- i  
            i++  
        }  
    }()  
  
    var j = 0  
    for j < 10 {  
        fmt.Println(<-channel)  
        j++  
    }  
}
```

In this example, a goroutine sends integers from 0 to 9 to the channel. The main function then receives these values from the channel and prints them. This mechanism not only facilitates communication but also synchronizes the two loops, as each print waits for a new value to be sent.

Buffered channels are a variation of channels that allow multiple values to be sent and stored without an immediate receiver.

```
func main() {  
    var channel = make(chan int, 2)  
    go func() {  
        var i = 0  
        for i < 10 {  
            channel <- i  
            i++  
        }  
    }()  
  
    var j = 0  
    for j < 10 {  
        fmt.Println(<-channel)  
        j++  
    }  
}
```

Here, the buffer size of 2 allows two values to be stored in the channel without requiring an immediate read.

Select

The `select` statement in Go provides a way to handle multiple channel operations. It waits until at least one of its cases can proceed and then executes it. This is particularly useful when you want to listen to multiple channels simultaneously.

```
func main() {
    var chan1 = make(chan int, 2)
    var chan2 = make(chan int, 2)
    var chan3 = make(chan int, 2)

    select {
    case v1 := <-chan1:
        fmt.Printf("Received %v from channel 1\n", v1)
    case v2 := <-chan2:
        fmt.Printf("Received %v from channel 2\n", v2)
    case chan3 <- 1:
        fmt.Println("Sent value to channel 3")
    default:
        fmt.Println("No one is ready to communicate")
    }
}
```

In this code, `select` listens on three channels. If any channel operation is ready, it will be executed; otherwise, it falls back to the default case.

Using `select`, we can also implement timeouts for receiving data. This can be done by creating a timeout channel with `time.After`.

```
func main() {
    var chan1 = make(chan int, 2)
    var timeout = time.After(5 * time.Second)

    select {
    case v1 := <-chan1:
        fmt.Printf("Received %v from channel 1\n", v1)
    case <-timeout:
        fmt.Println("Timeout")
    }
}
```

In this example, the program waits for data from `chan1`. If no data is received within 5 seconds, it proceeds with the timeout case, printing "Timeout".

When multiple goroutines need to access shared resources, data races can occur. Go provides `sync.Mutex` to lock data, ensuring that only one goroutine accesses it at a time.

```
var mu sync.Mutex
var idx int

func Increment() {
    mu.Lock()
    defer mu.Unlock()
    idx++
}
```

The `Increment` function locks `mu` before updating `idx`, preventing other goroutines from accessing `idx` until it's unlocked. By calling `defer mu.Unlock()`, we ensure that `mu` is unlocked at the end of the function, regardless of how the function exits.

Chapter 6

Web Front-end Programming with Vue.js

6.1 HTML

HTML, short for HyperText Markup Language, has been the standard language for creating web pages since its inception in 1991. As of today, HTML 5.2 represents the latest iteration, incorporating decades of evolution and refinement. Unlike procedural programming languages, HTML is a description language. It defines the structure and content of web pages but doesn't control how actions are executed on them.

HTML uses a system of tags to structure text, images, links, and other elements into meaningful, organized content that can be displayed in a web browser.

At its core, an HTML element consists of three parts: an opening tag, content, and a closing tag. These elements form the building blocks of HTML.

```
<tagname>  
  content  
</tagname>
```

Every HTML document follows a basic structure, starting with a declaration of the document type (`!DOCTYPE html`), followed by the `html` root element. Inside the `html` tag, there are two main sections: `head` and `body`. The `head` contains metadata about the document, while the `body` contains the visible content. An example of a simple HTML document:

```
<head>  
  <title>Page Title</title>  
</head>  
<body>  
  <h1>Heading</h1>  
  <p>A paragraph.</p>  
  <p>A longer paragraph.</p>  
</body>  
</html>
```

In this document: the `<title>` tag specifies the title of the web page, visible on the browser tab; `<h1>` creates a large, bold heading; `<p>` defines paragraphs.

HTML elements

HTML offers a variety of elements to structure and display different types of content. Here are some frequently used elements:

- *Headings:* `<h1>` to `<h6>` define headings, with `<h1>` being the largest and most important.
- *Paragraphs:* `<p>` creates blocks of text.

- *Line Breaks:* `<br>` introduces a line break without a closing tag.
- *Links:* `<a>` defines hyperlinks.
- *Images:* `<img>` embeds images.
- *Lists:* `<ul>` for unordered lists, `<ol>` for ordered lists, and `<li>` for list items.
- *Tables:* `<table>` organizes data into rows (`<tr>`) and cells (`<td>` or `<th>` for headers).

Attributes

Links in HTML are created using the `<a>` tag, which includes an `href` attribute to specify the destination URL:

```
<a href="http://gamificationlab.uniroma1.it/wasa">Our course</a>
```

Attributes like `href` provide additional information about elements and follow a `name="value"` format. They are always included in the opening tag of an element.

The `<img>` tag is used to embed images in a web page. It requires attributes such as `src` for the image's path, `alt` for alternate text (displayed if the image fails to load), and optionally `width` or `height` for size adjustments.

```

```

This code displays an image with a width of 250 pixels and the alternate text "Sapienza's logo".

Tables are a way to organize data in rows and columns. The `<table>` element contains `<tr>` (table rows), `<td>` (table data cells), and `<th>` (table headers):

```
<table>
  <tr>
    <th>Room</th>
    <th>Places</th>
  </tr>
  <tr>
    <td>1L</td>
    <td>120</td>
  </tr>
  <tr>
    <td>2L</td>
    <td>96</td>
  </tr>
</table>
```

This example creates a table with two columns labeled "Room" and "Places", listing details for two rooms.

HTML supports both unordered and ordered lists. Unordered lists use `<ul>`, while ordered lists use `<ol>`. Each list item is wrapped in `<li>`:

```
<ul>
  <li>Alice</li>
  <li>Bob</li>
</ul>

<ol>
  <li>Alice</li>
  <li>Bob</li>
</ol>
```

Tags can also be nested. For example, an image inside a hyperlink can be structured as:

```

<table>
  <tr>
    <td>
      <a href="https://www.uniroma1.it">
        
      </a>
    </td>
  </tr>
</table>

```

6.2 CSS

Cascading Style Sheets, or CSS, is a language used to control the presentation of web pages. It works alongside HTML, defining the appearance of elements by specifying layouts, colors, fonts, and more. The “cascading” aspect of CSS means that styles are applied hierarchically, with parent elements influencing their children unless explicitly overridden.

CSS styles can be included in web pages in three main ways:

1. Inline, directly within an HTML element using the style attribute.
2. Internal, within a <style> element in the <head> section of the HTML document.
3. External, through a linked CSS file that can style multiple HTML pages.

This flexibility allows developers to create reusable, maintainable, and scalable designs.

Inline CSS

Inline CSS applies styles directly to a single HTML element using the style attribute. For instance:

```

<!DOCTYPE html>
<html>
  <body>
    <h1 style="color:blue;">A Blue Heading</h1>
    <p style="color:red;">A red paragraph.</p>
  </body>
</html>

```

Here, the <h1> element has its color set to blue, while the <p> element is styled in red. Inline CSS is useful for quick customizations but is not recommended for large-scale projects due to its lack of reusability and cluttered appearance.

Internal CSS

Internal CSS defines styles for an entire HTML document, placed within a <style> block in the <head> section. For example:

```

<!DOCTYPE html>
<html>
  <head>
    <style>
      body { background-color: silver; }
      h1 { color: blue; }
      p { color: red; }
    </style>
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>

```

This example sets the background color of the page to silver, the heading text to blue, and paragraph text to red. Internal CSS centralizes styles but limits them to the specific page.

External CSS

External CSS enables developers to define styles in a separate file, which can be linked to multiple HTML documents. This approach enhances maintainability and consistency across pages.

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>This is a heading</h1>
    <p>This is a paragraph.</p>
  </body>
</html>
```

Contents of styles.css:

```
body {
  background-color: silver;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

The external file `styles.css` applies the same styles as the internal example, but its separation from the HTML file ensures cleaner and more modular code.

Overriding

CSS styles can be overridden based on specificity and location:

- Inline styles take precedence over both internal and external styles.
- Internal styles override external styles for the same element.
- Browser default styles are the least specific and can be overridden by any CSS rule.

```
<head>
  <style>
    h4 { color: green; }
  </style>
</head>
<body>
  <h4>This is green</h4>
  <h4 style="color:cyan;">This is cyan</h4>
  <h4 style="font-family:sans-serif;">Green sans</h4>
</body>
```

The second `<h4>` element is styled cyan due to its inline style attribute, overriding the internal rule that sets all `<h4>` elements to green.

Selectors

A basic CSS rule consists of a selector, which targets elements, and declarations, which define the styles to apply. Each declaration includes a property and a value:

```
h1 {
  color: blue;
  font-size: 12px;
}
```

Here: the selector `h1` targets all `<h1>` elements; the declarations set the color to blue and the font size to 12 pixels. CSS selectors allow developers to target elements precisely:

- Type selectors target element names (e.g., `p`).
- ID selectors use a hash (`#`) to target a specific element with an ID (e.g., `#hot`).
- Class selectors use a period (`.`) to target elements with a class (e.g., `.xxl`).

```
<head>
  <style>
    p { text-align: center; }
    #hot { color: red; }
    .xxl { font-size: 300%; }
  </style>
</head>
<body>
  <p>Just centered</p>
  <p id="hot">Centered red</p>
  <p class="xxl">Centered large</p>
</body>
```

CSS allows grouping multiple selectors to apply the same styles:

```
h1, h2, p {
  text-align: center;
  color: red;
}
```

The universal selector (`*`) applies styles to all elements:

```
* {
  text-align: center;
  color: blue;
}
```

Combinators refine the relationship between elements:

1. Descendant selector () selects all descendants of a specified element:

```
div p { color: red; }
```

This targets `<p>` elements within `<div>` elements.

2. Child selector (`>`) selects direct children only:

```
div > p { color: red; }
```

3. Adjacent sibling selector (`+`) selects the immediate sibling:

```
div + p { color: red; }
```

4. General sibling selector (`~`) selects all siblings:

```
div ~ p { color: red; }
```

Each combinator narrows the scope of the selector to achieve precise styling.

6.3 Javascript

JavaScript is a versatile programming language widely used for web development. Its dynamic capabilities allow developers to create interactive and responsive web pages. It is a just-in-time (JIT) compiled language, which means the code is compiled during execution rather than beforehand, enabling fast execution. While primarily executed in web browsers, JavaScript is also used in server environments equipped with JS engines, such as Node.js.

One of JavaScript's most powerful features is its ability to interact with and manipulate the Document Object Model (DOM), a representation of the web page structure. This feature enables developers to dynamically update web content, styles, and even user interfaces.

Types and variables

JavaScript supports various data types, including strings, numbers, booleans, and objects (which also encompass arrays). Its dynamic typing means that a variable's type can change during its lifetime. Additionally, JavaScript is weakly typed, which means it performs implicit type casting depending on the operation.

For instance:

```
let example = "Text"; // example is a string
example = 42;         // example is now a number
```

JavaScript provides several ways to declare variables, each with unique scoping rules:

- `var`: Function-scoped and can be redeclared.
- `let`: Block-scoped and cannot be redeclared within the same block.
- `const`: Block-scoped and immutable once assigned.

```
let greeting = "Hello";
greeting += " World"; // Now "Hello World"

const number = 10;
// number = 20; // This would throw an error as `const` cannot be reassigned.
```

It's worth noting that assigning variables without declaration, like `x = "foo";`, creates a global variable and is considered outdated.

Objects and arrays

Objects are mutable collections of key-value pairs, where keys are typically strings, and values can be any JavaScript type. For example:

```
let person = {
  name: "Alice",
  age: 30
};
person.age = 31; // The age property is updated
```

Arrays in JavaScript are a special type of object. They store data sequentially, but their indexes are essentially strings converted from integers. Here's an example:

```
const fruits = ["apple", "banana", "cherry"];
fruits[3] = "date"; // Adds "date" at index 3
console.log(fruits[0]); // Outputs: apple
```

Functions

Functions are first-class citizens in JavaScript. This means they can be assigned to variables, passed as arguments, and returned from other functions. Functions come in various forms, including traditional function declarations and arrow functions.

```
function multiply(x) {  
  return x * 2;  
}  
console.log(multiply(5)); // Outputs: 10
```

Introduced in ES6, arrow functions offer a concise syntax:

```
const triple = (x) => x * 3;  
console.log(triple(4)); // Outputs: 12
```

JavaScript functions can capture variables from their surrounding scope, forming closures. For example:

```
function outer() {  
  const value = 42;  
  return function inner() {  
    return value;  
  };  
}  
const innerFunc = outer();  
console.log(innerFunc()); // Outputs: 42
```

Loops

JavaScript offers several looping mechanisms:

- for loops for general iteration.
- for...in to iterate over object properties.
- for...of to iterate over iterable values, like arrays.
- the .forEach() method for arrays, which applies a function to each element.

```
const items = ["a", "b", "c"];  
items.forEach((item) => console.log(item));  
// Outputs: a, b, c
```

HTML and DOM

JavaScript can dynamically modify HTML and CSS. For instance, you can use:

- innerHTML to set or retrieve HTML content.
- document.write() to output directly into the document.
- console.log() for debugging.

```
<p id="demo"></p>  
<script>  
document.getElementById("demo").innerHTML = "Hello, World!";  
</script>
```

```
<p id="demo">This text will turn red.</p>  
<script>  
document.getElementById("demo").style.color = "red";  
</script>
```

Modules

Modules enable code organization and reusability. The `export` keyword marks variables or functions for external use, while `import` allows their inclusion in other files.

```
// File: module.js
export const greeting = "Hello";

// File: main.js
import { greeting } from './module.js';
console.log(greeting); // Outputs: Hello
```

Async-await

JavaScript, by its nature, operates in a single-threaded environment. This means only one task can be executed at a time. However, modern web applications demand responsiveness and the ability to perform multiple operations simultaneously. This is where asynchronous programming becomes crucial. Asynchronous JavaScript allows developers to write code that doesn't block the execution of other operations, ensuring a seamless user experience.

One of the foundational concepts in asynchronous JavaScript is the Promise. A Promise is essentially an object that acts as a placeholder for a value that will be available in the future. When a function returns a Promise, it signals that the result of its execution will be determined later, allowing the program to continue running other tasks in the meantime.

Promises operate in three distinct states:

1. *Pending*: The initial state, where the operation has neither completed nor failed.
2. *Fulfilled*: The operation successfully completed, and the resulting value is now available.
3. *Rejected*: The operation failed, and an error has been encountered.

Using Promises involves chaining methods to handle success and failure. For example, suppose we have a function `doSomething()` that returns a Promise. The syntax to handle its resolution and rejection might look like this:

```
doSomething()
  .then((result) => {
    // Handle the successful result
  })
  .catch((error) => {
    // Handle any errors
  });
```

Here, the `then` method is executed when the Promise is fulfilled, and the `catch` method is called if it is rejected.

It's essential to understand that the execution flow in JavaScript does not wait for Promises to resolve. For instance:

```
let txt;
doSomething()
  .then((result) => {
    txt = result;
  });

// This alert may run before txt is set
alert(txt);
```

In this scenario, the `alert` function might execute before the Promise has resolved and set the value of `txt`, illustrating the asynchronous nature of Promises.

To simplify working with Promises, JavaScript provides the `async-await` syntax. This syntactic sugar makes asynchronous code resemble synchronous code, improving readability without altering its asynchronous behavior.

Functions that use the async-await pattern must be declared with the `async` keyword. Within these functions, the `await` keyword pauses execution until the Promise resolves. However, the overall behavior remains asynchronous, and developers should remain conscious of this.

Let's compare traditional Promise chaining with the async-await syntax:

- *Using Promises:*

```
function buttonClicked() {
  doSomething()
    .then((result) => {
      alert(result);
    })
    .catch((error) => {
      // Handle the error
    });
}
```

- *Using Async-Await:*

```
async function buttonClicked() {
  try {
    let txt = await doSomething();
    alert(txt);
  } catch (error) {
    // Handle the error
  }
}
```

The async-await pattern reduces nesting and improves the readability of code, making it easier to debug and maintain.

6.4 Vue.js

Frameworks have become indispensable tools in web development. Unlike libraries, which provide pre-written code to perform specific tasks, frameworks impose a structure and a methodology for building applications. This "inversion of control" means the framework takes charge of the application's flow, enabling extensibility while remaining unmodifiable by the user.

Why do we need frameworks? The answer lies in JavaScript's origin. It was never designed with full-fledged application development in mind. Although the language has evolved, it still lacks built-in solutions for complex tasks, leaving developers to grapple with repetitive "boilerplate" code. Frameworks like Vue.js, React, and Angular bridge this gap, streamlining development.

What Is Vue.js? Vue.js, pronounced as "view," is a progressive JavaScript framework designed for building user interfaces. It builds upon standard HTML, CSS, and JavaScript, offering a declarative and component-based programming model. This approach simplifies the development of both simple and complex applications. Whether you're crafting a small interactive widget or a comprehensive single-page application (SPA), Vue.js provides the tools you need.

Vue.js stands out for two key features: declarative rendering and reactivity. Declarative rendering allows developers to specify how the UI should look based on the application state, while reactivity ensures that changes in the underlying data automatically update the UI.

For example:

```
const app = Vue.createApp({
  data() {
    return { message: "Hello, Vue!" };
  },
});
app.mount('#app');
```

In the associated HTML:

```
<div id="app">{{ message }}</div>
```

When the message changes, the content inside the `<div>` updates automatically, thanks to Vue's reactivity.

Vue.js can be used in multiple ways:

- As a standalone library for adding interactivity to existing web pages.
- Embedded as web components in legacy applications.
- To build SPAs that run entirely on the client side.
- For full-stack/server-side rendering, where both server and client collaborate.
- For static site generation, producing pre-rendered static files without requiring JavaScript execution on the client.

Template

At the heart of Vue's template syntax is text interpolation, which utilizes "Mustache" syntax—double curly braces—to bind variables to the HTML content dynamically. For example:

```
<span>Message: {{ msg }}</span>
```

If the variable `msg` contains the string `"Hello world!"`, the rendered HTML becomes:

```
<span>Message: Hello world!</span>
```

Vue automatically escapes HTML within variables to ensure security. For instance, if `msg` contains `<b>Hello</b>`, the output will display as:

```
<span>Message: &lt;b&gt;Hello&lt;/b&gt;</span>
```

This behavior prevents cross-site scripting (XSS) vulnerabilities.

Vue templates support JavaScript expressions directly within the interpolation syntax. These expressions allow dynamic content generation, such as:

```
{{ number + 1 }}
{{ ok ? 'YES' : 'NO' }}
{{ message.split('').reverse().join('') }}
```

However, Vue restricts the use of flow control statements like `if` or variable declarations within these expressions. For example:

```
<!-- These will not work -->
{{ var a = 1 }}
{{ if (ok) { return message } }}
```

Calling JavaScript functions within templates is supported. For instance:

```
{{ formatDate(date) }}
```

It is essential to note that Vue re-executes these functions whenever the template is updated, so they should remain efficient.

Vue extends HTML with custom attributes called directives. These directives allow for powerful template functionalities and are prefixed with `v-`.

Some key directives include:

1. `v-text`: Updates the text content of an element. For example:

```
<span v-text="msg"></span>
```

If `msg` changes, the content of the `<span>` updates dynamically.

2. `v-html`: Dynamically updates the HTML content but does not escape it—use it cautiously to avoid security risks.

3. `v-show` and `v-if`: Control element visibility and rendering, respectively. While `v-if` removes the element entirely from the DOM, `v-show` toggles visibility with CSS.
4. `v-for`: Enables iteration over arrays or objects to generate repetitive structures. For instance:

```
<div v-for="item in items">{{ item }}</div>
```

If `items` contains `['a', 'b']`, the rendered HTML becomes:

```
<div>a</div>
<div>b</div>
```

5. `v-bind`: Dynamically binds attributes to variables. For example:

```
<span v-bind:id="dynamicId"></span>
```

can be shortened as:

```
<span :id="dynamicId"></span>
```

6. `v-model`: Creates a two-way binding between an input element and a JavaScript variable. For example:

```
<input type="text" v-model="name" />
```

This directive applies to `<input>`, `<select>`, `<textarea>`, and Vue components.

Vue facilitates interaction with user events via the `v-on` directive. For example:

```
<span v-on:click="alert('Hello!')">World</span>
```

The shorthand syntax `@click` achieves the same result:

```
<span @click="alert('Hello!')">World</span>
```

Vue supports all standard HTML events and custom component events.

Vue provides two primary mechanisms to manage conditional content:

1. `v-if`:

```
<p v-if="showMessage">This appears if showMessage is true.</p>
```

2. `v-show`:

```
<p v-show="showMessage">This is visible if showMessage is true.</p>
```

Unlike `v-if`, `v-show` does not remove the element from the DOM but simply toggles its visibility.

Stepper

Imagine building a stepper, a simple counter with buttons to increment and decrement its value. This stepper will consist of two buttons, `"-"` and `"+"`, with a number displayed between them.

The initial structure looks like this:

```
<template>
  <button>-</button>
  0
  <button>+</button>
</template>
```

To make the counter functional, we need a variable to store its value. Vue.js introduces a reactive state using the `data()` function, which returns an object containing all reactive variables:

```
<script>
export default {
  data() {
    return { counter: 0 };
  },
};
</script>
```

Vue.js provides directives like `click` to listen for events. We can update the counter value when a button is clicked:

```
<template>
  <button @click="counter--">-</button>
  {{ counter }}
  <button @click="counter++">+</button>
</template>
```

Instead of directly modifying the counter, we can define methods for better code organization:

```
<script>
export default {
  data() {
    return { counter: 0 };
  },
  methods: {
    incr() {
      this.counter++;
    },
    decr() {
      this.counter--;
    },
  },
};
</script>
```

In the template, these methods are invoked:

```
<template>
  <button @click="decr()">-</button>
  {{ counter }}
  <button @click="incr()">+</button>
</template>
```

Vue.js allows dynamic styling based on the application's state. For instance, we can change the counter's color to red if the value is negative:

```
<template>
  <button @click="decr()">-</button>
  <span :style="dCol">{{ counter }}</span>
  <button @click="incr()">+</button>
</template>
```

The `:style` directive binds the `dCol` variable to the `<span>` element's style. The reactive state and methods manage this variable:

```
data() {
  return {
    counter: 0,
    dCol: 'color:black;',
  },
},
methods: {
```

```

incr() {
  this.counter++;
  if (this.counter >= 0) {
    this.dCol = 'color:black;';
  }
},
decr() {
  this.counter--;
  if (this.counter < 0) {
    this.dCol = 'color:red;';
  }
},
}

```

To simplify dynamic style handling, we can use computed properties. These properties automatically update based on their dependencies:

```

computed: {
  stepperStyle() {
    let color = this.counter < 0 ? 'red' : 'black';
    return `color:${color}; font-size:${Math.abs(this.counter)}em;`;
  },
},

```

The template then uses this property:

```

<template>
  <span :style="stepperStyle">{{ counter }}</span>
</template>

```

To promote reusability, we can encapsulate the stepper into its own component, `Stepper.vue`. This modular design allows it to be used across different parts of the application:

```

<template>
  <div>
    <button @click="decr()">-</button>
    <span :style="stepperStyle">{{ counter }}</span>
    <button @click="incr()">+</button>
  </div>
</template>

<script>
export default {
  data() {
    return { counter: 0 };
  },
  methods: {
    incr() {
      this.counter++;
    },
    decr() {
      this.counter--;
    },
  },
  computed: {
    stepperStyle() {
      let color = this.counter < 0 ? 'red' : 'black';
      return `color:${color}; font-size:${Math.abs(this.counter)}em;`;
    },
  },
};
</script>

```

Vue's `v-for` directive allows dynamic rendering of multiple components. For example, if we want to manage counters for different courses:

```
<template>
  <div>
    <Stepper v-for="course in courses" :key="course" :title="course" />
  </div>
</template>

<script>
import Stepper from './Stepper.vue';
export default {
  components: { Stepper },
  data() {
    return {
      courses: ['Math', 'Physics', 'Chemistry', 'Biology'],
    };
  },
};
</script>
```

The `v-model` directive binds input fields to variables, enabling two-way data binding. This can be used to dynamically add courses:

```
<template>
  <input v-model="newCourse" placeholder="Enter course name" />
  <button @click="addCourse" :disabled="!newCourse">Add</button>
</template>

<script>
export default {
  data() {
    return {
      courses: [],
      newCourse: '',
    };
  },
  methods: {
    addCourse() {
      if (this.newCourse) {
        this.courses.push(this.newCourse);
        this.newCourse = '';
      }
    },
  },
};
</script>
```

Options and Composition

Vue.js provides two programming styles: the Options API and the Composition API. The Options API, often considered more beginner-friendly, organizes code into options like `data`, `methods`, and `computed`. On the other hand, the Composition API introduces a more functional approach, allowing developers to better manage complex logic.

A simple Vue.js app combines JavaScript, HTML, and CSS. Here's an example of a basic counter:

```
import { createApp } from 'vue';

createApp({
  data() {
```

```

    return { count: 0 };
  },
}).mount('#app');

<div id="app">
  <button @click="count++">Count is: {{ count }}</button>
</div>

```

Single-File Components (SFCs)

A powerful feature of Vue.js is its single-file components (SFCs). These components encapsulate JavaScript, HTML, and CSS into a single `.vue` file. This modular approach promotes reusability and maintainability.

```

<script>
export default {
  data() {
    return { count: 0 };
  },
};
</script>

<template>
  <button @click="count++">Count is: {{ count }}</button>
</template>

<style scoped>
button {
  font-weight: bold;
}
</style>

```

The `scoped` keyword ensures the styles apply only to this component.

Single-Page Applications (SPAs)

An SPA is a web application that interacts with the user by dynamically updating the current page rather than loading new pages from the server. In Vue.js, the `index.html` file serves as the foundation for the SPA:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <title>Example App</title>
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"></script>
</body>
</html>

```

The `main.js` file initializes Vue.js and mounts it to the `#app` element:

```

import { createApp } from 'vue';
import App from './App.vue';

createApp(App).mount('#app');

```

Reactive

Reactive programming is a declarative paradigm centered around data streams and the propagation of changes. It offers a way to automatically adjust dependent values whenever the source values change. For instance, if we have a reactive variable `b` defined as `b = a + 1`, updating the value of `a` will immediately update `b` as well.

In an imperative language, the relationship between variables is static unless explicitly updated. For example:

```
// Imperative programming
let a = 2;
let b = a + 1;
console.log(b); // Outputs: 3
a = 1;
console.log(b); // Still outputs: 3
```

In reactive programming, dependencies are tracked automatically:

```
// Reactive programming
let a = 2;
reactive let b = a + 1;
console.log(b); // Outputs: 3
a = 1;
console.log(b); // Outputs: 2
```

This dynamic nature is what makes reactive programming a powerful tool for modern applications.

In Vue.js, each component maintains a reactive state. This is declared within the `data()` function, and Vue wraps the returned object in its reactive system.

Here's an example of a basic reactive state:

```
export default {
  data() {
    return {
      count: 0,
    };
  },
};
```

All reactive variables must be declared within the `data()` function.

Nested objects are deeply reactive, meaning changes within their properties are tracked.

Reserved prefixes like `$` or `_` should not be used for variable names as they are reserved for Vue's internal use.

You can interact with reactive variables either programmatically in JavaScript or directly within templates. In JavaScript, you use `this` to refer to the reactive state:

```
export default {
  data() {
    return { count: 1 };
  },
  mounted() {
    console.log(this.count); // Outputs: 1
    this.count = 2; // Updates the state and triggers reactivity
  },
};
```

In templates, you don't need the `this` keyword:

```
<span>Count: {{ count }}</span>
<div v-text="count"></div>
```

Vue allows you to define computed properties that automatically update when their dependencies change. These properties are useful for deriving state without duplicating logic.

Here's an example:


```
export default {
  data() {
    return { count: 0 };
  },
  computed: {
    realCount() {
      return this.count + 1;
    },
  },
};
```

In the template:

```
<span>{{ realCount }}</span>
```

The value of `realCount` updates whenever `count` changes.

Vue automatically updates the DOM when the reactive state changes, though these updates are buffered and applied in batches during the next “tick” (update cycle). If you need to access the updated DOM immediately after state changes, use the `nextTick` function:

```
import { nextTick } from 'vue';
export default {
  methods: {
    increment() {
      this.count++;
      nextTick(() => {
        // Access the updated DOM here
      });
    },
  },
};
```

Components can include methods for encapsulating logic. These methods can be called from JavaScript or directly within templates. For instance:

```
export default {
  data() {
    return { count: 0 };
  },
  methods: {
    increment() {
      this.count++;
    },
  },
};
```

In the template:

```
<button @click="increment">{{ count }}</button>
```

Methods should not be defined using arrow functions as they do not bind `this` to the component instance:

```
export default {
  methods: {
    increment: () => {
      // BAD: No access to `this`
    },
  },
};
```

Vue components go through a series of lifecycle stages, such as being created, mounted, updated, and unmounted. These stages allow you to execute specific logic at precise moments. For example:

```
export default {
  mounted() {
    console.log('The component is now mounted.');
```

For an in-depth guide, refer to Vue's Lifecycle Documentation.

Routing

The SPA architecture relies on a server to deliver the necessary JavaScript, HTML, and CSS files. Once loaded, the SPA communicates with the server via APIs to send and receive data. This seamless interaction forms the backbone of SPAs, making frameworks like Vue.js highly effective in creating dynamic and responsive applications.

In an SPA, navigation between different views or pages occurs without reloading the browser. This is where a router comes into play. The router allows users to navigate through the application using URLs, ensuring that each view has a unique URL. This unique identification of views supports features like bookmarking and sharing.

Additionally, routers enable the use of URL or query parameters, which can be passed dynamically. For instance, a route like `/user/:id` allows the application to display different content based on the `id` parameter.

To understand how Vue Router facilitates navigation, let's explore its core components:

1. **Routes:** These define the mapping between URLs and components. Each route specifies a path and the corresponding component to render.
2. **RouterView:** This is a placeholder in the main Vue component. It dynamically renders the component associated with the current URL.
3. **RouterLink:** This component creates navigational links. Clicking a RouterLink updates the URL and renders the associated component in the RouterView.

For example, a link defined as `<RouterLink to="/link1">Page 1</RouterLink>` will replace the content in the RouterView with the component registered for `/link1` when clicked.

Vue Router manages navigation history using a stack. This allows users to navigate forward and backward through their browsing history. The following actions can manipulate the stack programmatically:

- **Push:** Adds a new view to the stack and switches to it.
- **Go:** Moves back or forward in the stack.
- **Replace:** Similar to Push, but replaces the current view instead of adding a new one.

Users can also navigate using browser controls like the back and forward buttons.

Here's a sample implementation of Vue Router in JavaScript:

```
import { createRouter, createWebHashHistory } from 'vue-router';
import HomeView from '../views/HomeView.vue';
import Page1View from '../views/Page1View.vue';
import Page2View from '../views/Page2View.vue';

const router = createRouter({
  history: createWebHashHistory(),
  routes: [
    { path: '/', component: HomeView },
    { path: '/link1', component: Page1View },
    { path: '/some/:id/link', component: Page2View },
  ],
});
export default router;
```

In this example, the `createRouter` function initializes the router with a set of routes. The history mode is set to `createWebHashHistory`, ensuring compatibility with browsers.

The main component integrates the router as follows:

```
<script setup>
import { RouterView } from 'vue-router';
</script>

<template>
  <header>App header</header>
  <main><RouterView /></main>
</template>
```

The `RouterView` dynamically displays the component associated with the current route.

Navigation can be implemented using `RouterLink` components:

```
<template>
  <p>
    <RouterLink to="/">Home</RouterLink>
    <RouterLink to="/link1">Page 1</RouterLink>
    <RouterLink to="/some/1/link">Page with Parameter</RouterLink>
  </p>
  <RouterView />
</template>
```

Alternatively, navigation can be triggered programmatically:

```
<script>
export default {
  methods: {
    goToPage(id) {
      this.$router.push('/some/' + id + '/link');
    },
  },
};
</script>

<template>
  <button @click="goToPage(1)">Go to Page 1</button>
</template>
```

Route parameters are accessible via `$route.params`:

```
<script>
export default {
  methods: {
    logParam() {
      console.log(this.$route.params.id);
    },
  },
};
</script>

<template>
  {{ $route.params.id }}
</template>
```

However, switching between routes with different parameter values does not reload the component. To handle such cases, you can watch for changes in route parameters:

```
export default {
  created() {
```

```

    this.$watch(
      () => this.$route.params,
      (newParams, oldParams) => {
        // React to route changes
      }
    );
  },
};

```

Ajax

AJAX, short for Asynchronous JavaScript and XML, is a set of technologies enabling web pages to update incrementally without reloading the entire page. This capability allows for more dynamic and responsive user experiences. Despite its name, AJAX isn't limited to XML data—it supports various formats, including JSON and plain text.

The core technologies powering AJAX include:

- HTML/XHTML/CSS for structuring and styling content.
- JavaScript for enabling interactivity.
- DOM (Document Object Model) for manipulating web page elements.
- XML/XSLT (optional) for data exchange.
- XMLHttpRequest (XHR) for making asynchronous requests.

These components together allow developers to build rich, interactive web applications.

The XMLHttpRequest object is the backbone of traditional AJAX. It enables web browsers to send and receive data asynchronously. Here's an example of how to use it:

```

let httpRequest;

function makeRequest() {
  httpRequest = new XMLHttpRequest();

  if (!httpRequest) {
    alert("Cannot create an XMLHttpRequest instance");
    return false;
  }

  httpRequest.onreadystatechange = alertContents;
  httpRequest.open("GET", "test.html");
  httpRequest.send();
}

function alertContents() {
  if (httpRequest.readyState === XMLHttpRequest.DONE) {
    if (httpRequest.status === 200) {
      alert(httpRequest.responseText);
    } else {
      alert("There was a problem with the request.");
    }
  }
}

```

While XMLHttpRequest is functional, it's verbose and lacks modern features, such as native support for promises or `async/await` syntax. Its outdated interface often makes it cumbersome to work with in contemporary JavaScript applications.

Modern JavaScript offers the Fetch API, a streamlined alternative to XMLHttpRequest. Fetch simplifies the process of making asynchronous requests and integrates well with modern features like promises. However, it still has limitations, such as a lack of built-in timeout handling, upload/download progress tracking, and backward compatibility with older browsers.

Axios

Axios is a popular JavaScript library built on top of XMLHttpRequest. It combines the robustness of XMLHttpRequest with modern JavaScript features, offering a simpler and more efficient way to handle asynchronous requests. Key features of Axios include:

- Promise-based API.
- Interceptors for requests and responses.
- Transformation of request and response data.
- Cancellation of requests.
- Automatic handling of JSON data.
- Cross-site request forgery (XSRF) protection.

These features make Axios a versatile tool for building modern web applications.

In Vue.js applications, Axios is often the go-to library for making HTTP requests. Here's an example of a GET request in a Vue.js component:

```
async refresh() {
  try {
    let response = await this.$axios.get("/");
    // response.data contains the JSON in the response body
  } catch (e) {
    console.error(e.toString());
  }
}
```

In this example:

- `this.$axios.get("/")` sends a GET request to the server's root URL.
- The `await` keyword ensures the code waits for the response before proceeding.
- Any errors are caught and logged to the console.

```
async sendData() {
  try {
    let response = await this.$axios.post("/some/api", {
      firstName: 'Fred',
      lastName: 'Flintstone',
    });
    // response.data contains the JSON in the response body
  } catch (e) {
    console.error(e.toString());
  }
}
```

In this case, the data object `{firstName: 'Fred', lastName: 'Flintstone'}` is sent in the request body to the endpoint `/some/api`.

A typical Axios response contains the following properties:

```
{
  data: {},           // The response payload
  status: 200,        // HTTP status code
  statusText: 'OK',   // Status message
  headers: {},        // HTTP headers
  config: {},         // Request configuration
  request: {}         // The original request object
}
```

This structure provides comprehensive details about the server's response, making it easy to debug and process data.

Axios interceptors allow developers to modify requests or responses before they are sent or received. This can be useful for adding authentication headers, logging, or error handling.

```
axios.interceptors.request.use(function (config) {  
  return config;  
}, function (error) {  
  return Promise.reject(error);  
});
```

This example processes the configuration object of every request. Errors are handled by rejecting the promise.

```
axios.interceptors.response.use(function (response) {  
  return response;  
}, function (error) {  
  return Promise.reject(error);  
});
```

Here, responses are processed before being passed to the calling code. Errors are handled similarly to requests.

Axios supports request cancellation using the `AbortController`. This feature is useful for terminating long-running or unnecessary requests:

```
const controller = new AbortController();  
  
axios.get('/foo/bar', {  
  signal: controller.signal  
}).then(function (response) {  
  // Handle response  
});  
  
// Cancel the request  
controller.abort();
```

In this example, `controller.abort()` terminates the request, ensuring no unnecessary network activity.

Chapter 7

Containers with Docker

7.1 Introduction to Linux Containers and Docker

Containers are a revolutionary technology in modern computing, designed to provide lightweight and efficient application isolation. They allow processes to run in restricted environments, making it easier to manage dependencies and improve scalability. Containers have become a cornerstone of cloud computing and microservices architecture, simplifying application deployment and maintenance.

Before understanding containers, it's essential to compare them with traditional virtualization.

In virtualization, there are two roles:

1. The host operating system runs on the actual hardware.
2. The guest operating systems run inside virtual machines.

A hypervisor manages the guest machines, translating their instructions to the host. Virtual machines are isolated, with virtual hardware devices. This isolation ensures that crashes or security incidents in one VM do not affect others. However, VMs require more resources because each VM has its own OS.

Containers offer a lighter alternative:

1. The host operating system runs directly on the hardware.
2. Containerized processes run in isolated environments.

Unlike VMs, containers share the host's kernel but operate in restricted environments. This makes containers faster and less resource-intensive, though with slightly reduced isolation compared to VMs. For example:

- Containerized processes cannot interact by default.
- The filesystem is isolated unless explicitly shared.

Containers have a rich history across various operating systems:

- 1982: UNIX introduced chroot for isolated filesystems.
- 2000: FreeBSD added Jails, and Solaris introduced Zones.
- 2005: Linux introduced LXC (Linux Containers).
- 2013: Docker popularized container usage, expanding to Windows and FreeBSD.

Linux containers leverage technologies like cgroups, introduced by Google in 2006 and published in 2008. Cgroups provide:

- *Resource Limiting*: Controlling CPU, memory, and I/O usage.
- *Prioritization*: Assigning higher priority to specific processes.
- *Accounting*: Measuring resource usage for billing or debugging.
- *Lifecycle Control*: Restarting on crashes or scheduling processes.

Namespace isolation

Namespaces are a fundamental feature of Linux containers. They create isolated environments for processes, preventing interaction across namespaces. Each namespace controls specific aspects:

- *Process ID (PID)*: Isolates process numbers.
- *Network*: Isolates network interfaces and configurations.
- *UTS/Hostname*: Isolates system identification.
- *Mount*: Controls access to filesystems.
- *IPC*: Isolates inter-process communication.
- *User*: Isolates user and group IDs.
- *Cgroup*: Manages resource constraints.

Linux namespaces draw inspiration from Plan 9, an experimental OS from Bell Labs, which also influenced UNIX and programming languages like C and Go.

Docker

Docker, introduced in 2013, is a set of tools for managing Linux containers. It has standardized containers through the Open Container Initiative (OCI), enabling interoperability with tools like Podman.

- *Container*: The isolated environment.
- *Container Image*: An archive of files and configurations to create containers.
- *Registry*: A repository for sharing and storing images (e.g., DockerHub).
- *Dockerfile*: A script defining how to build an image.
- *Volume*: An external storage area accessible to containers.

By default, containers use ephemeral storage. However, volumes can be mounted to preserve data across container lifecycles. For instance:

```
docker run -v /host/path:/container/path myimage
```

This command mounts `/host/path` as `/container/path` inside the container.

Container images are created using a Dockerfile, which defines the setup instructions. Each instruction forms a “layer,” enabling caching and reuse. For example:

```
FROM debian:stable
RUN apt-get update && apt-get install -y python3
```

- *Base Image*: Typically includes essential tools (e.g., `debian:stable` or `python:3`).
- *Layers*: Incremental changes on top of the base image.

This modularity allows efficient sharing and distribution of images.

Containers in the cloud

A key advantage of containers is their portability. A container built on one Linux platform can run on any other platform that supports cgroups and container-related technologies. This feature has made containers indispensable for cloud computing, where they enable consistent application deployment across diverse environments.

7.2 Running Your First Container

Once an image is ready, running a container is straightforward. For example:

```
docker run -it --rm debian:bullseye
```

This command:

- `-it`: Starts the container interactively.
- `--rm`: Automatically removes the container after it stops.
- `debian:bullseye`: Specifies the image to use.

The container will launch with an isolated environment based on the Debian Bullseye distribution.